vkGSplat: Is 3D Gaussian Splatting a Good Integration Layer for DLSS-Class Reconstruction?

vkGSplat research note

Started May 5, 2026

1 Executive Thesis

The answer is **qualified yes**. 3D Gaussian Splatting is a promising way to integrate Monte Carlo denoising, radiance caching, temporal anti-aliasing, temporal super-resolution, supersampling, frame interpolation, and learned reconstruction, but only if vkGSplat treats 3DGS as a *shared persistent reconstruction state*, not as a standalone renderer or a bag of unrelated tricks.

The central claim to test is:

> A single explicit 3D/4D Gaussian state can carry geometry, visibility, motion, radiance, subpixel coverage, variance, and confidence across frames. If that state can feed multiple reconstruction tasks with less ghosting, less shimmer, and lower variance than separate image-space systems, then 3DGS is a good integration layer for vkGSplat.

The software-stack consequence is important: 3DGS is *not* the whole renderer, and DLSS-style reconstruction is *not* the acquisition stage. vkGSplat should keep software renderers as replaceable acquisition producers. A CPU ray tracer, an Apple Metal backend, a future CUDA software ray tracer, a NVDIFFRAST-style CUDA triangle rasterizer, and a GraphDECO/GSPLAT Gaussian rasterizer should all be allowed to produce the same seed-buffer contract. The 3DGS/DLSS-class research layer begins after that boundary: it consumes color, depth, primitive identity, motion, variance, and auxiliary signals, then performs history, denoising, super-resolution, frame generation, and optional neural residual reconstruction.

The core framework therefore does *not* require massive training. The initial stack is analytic and systems-oriented: software acquisition, seed buffer extraction, reprojection, depth/identity rejection, SVGF-style denoising, temporal accumulation, Gaussian state updates, and CUDA tile scheduling. Training appears only as an optional later residual model. The research bet is that explicit geometry, motion, variance, identity, and confidence should reduce what a neural model must learn, not force vkGSplat to replicate a proprietary DLSS-scale training pipeline.

This does *not* mean vkGSplat can directly clone DLSS 4.5 or the announced DLSS 5 target. NVIDIA's DLSS 4.5 combines a second-generation transformer Super Resolution model, Dynamic Multi Frame Generation, 6X Multi Frame Generation, Reflex, mature engine integration, optical flow, training data, and heavy production tuning [27, 28]. NVIDIA has also announced DLSS 5 as a real-time neural rendering system that uses game color and motion-vector inputs to synthesize photoreal lighting and material detail while remaining anchored to the game scene [26]. vkGSplat's stronger research position is different:

> DLSS reconstructs from images plus engine buffers. vkGSplat can reconstruct from images plus explicit Gaussian scene, motion, and radiance state.

2 Training Requirement

vkGSplat should be evaluated in three increasingly expensive modes:

1. **No-training core.** The required baseline is deterministic: seed acquisition, motion-map derivation, history rejection, temporal accumulation, denoising, Gaussian update, and Gaussian projection are implemented as explicit CPU/Metal/CUDA kernels. This mode must produce useful results before any neural model is introduced.

2. **Small calibrated residual.** If the analytic path exposes stable Gaussian feature planes, a compact residual network may learn bounded corrections for high-frequency shading, specular residuals, or difficult disocclusions. This model is optional and should be tested against an equal-size image-only model.

3. **DLSS-class product model.** A large model trained across many games/scenes is out of scope for the core framework. It belongs only to a future productization path, not to the proof that the software stack and Gaussian state are useful.

This distinction is central. A failure to match DLSS product quality does not falsify vkGSplat. The falsifiable core question is whether explicit acquisition-independent Gaussian state improves reconstruction quality, stability, or cost before large-scale training enters the loop.

3 What "Good Integration Layer" Means

3DGS is useful here only if it satisfies four criteria:

1. **Shared state:** the same Gaussian state helps more than one feature: TAA, super-resolution, denoising, radiance caching, and interpolation.

2. **Better address space:** history is attached to world, object, or path evidence, not only previous-frame pixels.

3. **Better uncertainty:** each Gaussian carries confidence, variance, visibility age, primitive identity, material identity, and motion residuals.

4. **Compute-friendly:** update, query, and splat operations map to tiled CUDA/Metal-style kernels without turning into a training system.

5. **Acquisition-independent:** the layer consumes the same seed buffers no matter whether the evidence came from CPU, Metal, CUDA software rasterization, CUDA ray traversal, or Gaussian splatting.

The answer becomes **no** if each feature needs its own incompatible Gaussian state, if the state ghosts more than image history, or if the cost beats neither a strong TAAU/SVGF baseline nor a small neural resolver.

The implementation focus is CUDA. CPU code is only a small oracle for debugging and expected-value checks; the research artifact should answer whether a shared Gaussian state is a good *GPU* integration layer.

4 Software-Stack Boundary

This note is a companion to the broader vkGSplat system paper in `papers/vkGSplat.tex`. The system paper argues for a compute-first Vulkan path for synthetic data generation: capture existing Vulkan/Wicked workloads, lower acquisition to CUDA-class compute, and preserve a stable seed-buffer ABI. This note deliberately starts after that boundary. It asks whether a persistent Gaussian state is the right reconstruction and reuse layer once seed evidence already exists. Therefore the two papers should cross-reference each other rather than duplicate material: the system paper owns the Vulkan/SDG/hardware-allocation argument, while this note owns the 3DGS reconstruction, tensorization, radiance-cache, and frame-generation algorithms.

The current vkGSplat stack should be read as two independent layers joined by a stable seed ABI:

Vulkan/Wicked command stream or captured scene state

$\downarrow$

software acquisition renderer

$\downarrow$

seed buffers

$\downarrow$

3DGS/DLSS-class reconstruction layer

The acquisition layer answers the question: "what did the camera observe this frame?" It may use rasterization, ray traversal, shadow/light visibility, or Gaussian splatting. Its job is to emit structured measurements, not final beauty frames. A seed frame should contain at least:

- linear color or radiance,
- depth and NDC depth,
- primitive/object/material identity,
- camera and previous-camera matrices,
- screen-space motion or enough state to derive it,
- optional normal, albedo, variance, sample count, and path metadata.

The reconstruction layer answers a different question: "how should these measurements be reused over space, time, and scale?" It owns temporal anti-aliasing, denoising, super-resolution, radiance-cache reuse, frame interpolation, Gaussian state maintenance, and any learned residual. It should not branch on whether the seed came from CPU, Metal, or CUDA acquisition.

This separation changes how to use existing CUDA renderers. NVDIFFRAST is a source-level reference for a CUDA triangle acquisition backend: triangle setup, bin rasterization, coarse rasterization, and fine rasterization can produce depth, primitive ID, barycentrics, and coverage [2]. The original GraphDECO CUDA rasterizer released with 3DGS provides the canonical Gaussian acquisition path [1]; GSPLAT adds a maintained library reference for packed/unpacked projection, tile intersections, flattened IDs, and front-to-back compositing [3]. None of these should leak private assumptions into the DLSS-class reconstruction layer. They are producers of the same seed ABI.

The clean test is whether a recorded seed buffer can be replayed through the reconstruction layer with no acquisition backend present. If it can, the stack is modular. If reconstruction code requires calling a particular renderer, the boundary has failed.

5 Relevant Research

5.1 Core 3DGS

3D Gaussian Splatting introduced explicit anisotropic Gaussian primitives with fast visibility-aware rendering for real-time radiance-field view synthesis [1]. For vkGSplat, the important lesson is not just fast splats. It is that an explicit primitive can be optimized, projected, sorted, blended, and inspected.

5.2 Anti-Aliasing and Scale-Aware Splatting

Mip-Splatting identifies sampling-rate changes, camera distance, and focal length as causes of 3DGS aliasing, then combines 3D smoothing with a 2D Mip/box-filter approximation [4]. Multi-Scale 3DGS, Analytic-Splatting, AAA-Gaussians, LOD-GS, and AA-Splat all strengthen the case that splats need footprint-aware filtering before they can be a robust temporal reconstruction layer [5, 6, 7, 8, 9].

5.3 Radiance Caching and Monte Carlo Reconstruction

GSCache is the closest direct precedent: it uses 3D Gaussian splatting as a multi-level path-space radiance cache for volume path tracing [10]. Neural Radiance Caching and Neural Control Variates are conceptual ancestors for learned or corrected variance reduction [11, 12]. SVGF is the practical image-space baseline for one-sample-per-pixel real-time path-traced reconstruction [13]; KPCN is the classic learned Monte Carlo denoising baseline using auxiliary buffers [14]. ReSTIR shows how spatiotemporal sample reuse and biased/unbiased estimators can coexist in a real-time renderer [15]. StochasticSplats and Unified Gaussian Primitives show that Monte Carlo estimators and Gaussian representations can be coupled more deeply than normal raster splatting [16, 17].

5.4 Temporal and Motion-Aware Gaussians

4DGS, Spacetime Gaussian Feature Splatting, 4D-Rotor Gaussian Splatting, Gaussian-Flow, GaussianFlow, MotionGS, and KF-GS all point toward Gaussians as motion-carrying state, not only static scene primitives [18, 19, 20, 21, 22, 23, 24]. RetimeGS is especially relevant because it targets continuous-time 4DGS retiming and ghost-free arbitrary-timestamp interpolation [25].

5.5 DLSS, FSR, and XeSS

DLSS, FSR, and XeSS set the product bar for temporal upscaling and frame generation. DLSS 4 uses transformer models for Super Resolution, DLAA, and Ray Reconstruction; DLSS 4.5 upgrades Super Resolution and adds Dynamic Multi Frame Generation / 6X MFG on supported hardware [29, 30, 27]. DLSS 5 raises the comparison bar again: the public target is no longer only upscaling and frame generation, but a neural renderer that infers lighting/material detail from game outputs while preserving temporal control [26]. XeSS is AI-based temporal super sampling using jittered color, motion vectors, and depth [38]. AMD FSR includes temporal upscaling, frame generation, and newer ray regeneration/radiance-caching branding [39].

6 Unified vkGSplat Architecture

6.1 One State, Many Views

The proposed integration layer is a persistent state:

$$G_j = (\mu_j, \Sigma_j, R_j, c_j, f_j, L_j, v_j, m_j, \tau_j, id_j, \dot{\mu}_j, \nu_j, r_j).$$

Where:

- μ_j, Σ_j, R_j are position, covariance, and orientation,
- c_j is reconstructed color or albedo-like appearance,
- f_j is a compact learned or hand-authored feature vector,
- L_j is radiance/cache content, possibly split by path depth,
- v_j is variance,
- m_j is sample mass,
- τ_j is age or last-seen time,
- id_j stores primitive/material/motion group identity,
- $\dot{\mu}_j$ is velocity,
- ν_j is visibility confidence,
- r_j is residual error, such as projected flow mismatch.

For implementation, split this into a shared core plus task-specific subslots:

$$G_j = (C_j, A_j^{taa}, A_j^{sr}, A_j^{rad}, A_j^{mot}, A_j^{nn}).$$

The shared core is:

$$C_j = (\mu_j, \Sigma_j, R_j, \bar{n}_j, id_j, m_j, \tau_j, \nu_j, r_j).$$

Task subslots are:

$$A_j^{taa} = (c_j, v_j), \quad A_j^{sr} = (c_j, \Sigma_{screen,j}, m_j), \quad A_j^{rad} = (L_j^0, \ldots, L_j^K, V_{L,j}^0, \ldots, V_{L,j}^K),$$

$$A_j^{mot} = (\dot{\mu}_j, \mu_j^0, \mu_j^1, R_j^0, R_j^1), \quad A_j^{nn} = f_j.$$

This decomposition is the compromise that makes the integration thesis testable. The address, visibility, covariance, identity, and confidence are shared. Payloads may differ by feature. If the payloads force incompatible cores, the unified-state idea fails.

The state exposes multiple projections:

- **TAA projection:** splat stable history into current pixels.
- **Super-resolution projection:** resolve target-resolution pixels from accumulated subpixel Gaussian evidence.

- **Radiance-cache query:** query indirect radiance at path hits.
- **Frame-interpolation projection:** evaluate state at t_α and splat an intermediate frame.
- **Neural feature projection:** render Gaussian confidence, variance, radiance, age, and motion residuals as inputs to a small learned resolver.

6.2 Pipeline

Per frame:

1. Acquire a low-resolution/low-spp seed frame from any backend that satisfies the seed ABI: current CPU fixture, Metal validation path, future CUDA ray/raster path, or optional Gaussian raster path.
2. Extract samples: color, depth, NDC depth, normal, primitive ID, material ID, motion vector, path contribution, and optional path depth.
3. Update the Gaussian state with visibility and confidence gates.
4. Produce task-specific outputs from the same state: TAA history, SR resolve, radiance cache, interpolation state, and neural features.
5. Compare against current-frame samples and references, then decay or invalidate stale Gaussians.

6.3 Why This Could Beat Separate Systems

Separate systems duplicate partial history:

- TAA stores image-space color history.
- SVGF stores moments and filtered radiance.
- Super-resolution stores subpixel history.
- Frame interpolation stores optical-flow history.
- Radiance caching stores lighting history.

The 3DGS hypothesis is that these are shadows of the same underlying state: where scene evidence was, how it moved, how visible it is, what radiance it carried, and how reliable it is. A unified Gaussian state is attractive because it can make that evidence explicit.

7 Formal Model and Algorithms

7.1 Observations

At frame t, vkGSplat receives a set of observations:

$$S_t = \{s_i^t\}_{i=1}^{N_t}.$$

Each observation can come from a visible pixel, a subpixel jitter sample, or a path-tracing event:

$$s_i^t = (x_i, n_i, c_i, L_i^0, \ldots, L_i^K, d_i, z_i, u_i, \rho_i, id_i, q_i, \sigma_i^2).$$

Where x_i is world position, n_i is normal, c_i is color, L_i^k is a radiance contribution for path-depth level k, d_i is depth, z_i is NDC depth, u_i is screen-space motion, ρ_i is roughness/material metadata, id_i is primitive/material/motion identity, q_i is sample quality, and σ_i^2 is a local variance estimate.

7.2 Sample Extraction

For a raster or ray-traced seed pixel $p = (p_x, p_y)$, unproject with the current inverse view-projection matrix:

$$x_i = \Pi_t^{-1}(p, z_i) = (VP_t)^{-1} \begin{bmatrix} 2(p_x + \frac{1}{2})/W - 1 \\ 2(p_y + \frac{1}{2})/H - 1 \\ z_i \\ 1 \end{bmatrix}.$$

After homogeneous divide, x_i becomes the world-space address for history. This matches the existing vkGSplat reprojection contract: depth plus camera matrices are already enough to derive a current-to-previous motion map. The Gaussian path simply stores the result in a persistent state.

For temporal super-resolution with scale factor $s \in \{2, 4\}$, the low-res pixel contributes to high-res coordinates:

$$P_i = s\,p + s\,\delta_t,$$

where δ_t is the subpixel jitter. The sample therefore updates both a world-space Gaussian and a subpixel coverage estimate.

For path-traced samples, the extraction point is the path event rather than the pixel center:

$$s_i^t = (x_\ell, n_\ell, \omega_o, \omega_i, \beta_\ell L_\ell, pdf_\ell, \ell, id_\ell),$$

where ℓ is path depth and β_ℓ is throughput. The first radiance-cache prototype should keep only $\ell = 1$ rough/diffuse indirect events so the experiment is hard to fool but still easy to debug.

7.3 Compatibility Gates

An observation updates a Gaussian only if it is compatible. Define:

$$W_{ij} = w_x\, w_n\, w_d\, w_{id}\, w_\rho\, w_v\, w_r\, w_q.$$

Spatial compatibility:

$$w_x = \exp\left(-\frac{1}{2}(x_i - \mu_j)^T \Sigma_j^{-1} (x_i - \mu_j)\right).$$

Normal compatibility:

$$w_n = \max(0, n_i \cdot \bar{n}_j)^{\gamma_n}.$$

Depth compatibility after projection:

$$w_d = \begin{cases} 1, & |\hat{d}_{ij} - d_i| < \epsilon_d, \\ 0, & \text{otherwise.} \end{cases}$$

Identity compatibility:

$$w_{id} = \begin{cases} 1, & id_i = id_j, \\ \eta_{id}, & id_i \neq id_j \text{ but material-compatible}, \\ 0, & \text{otherwise}. \end{cases}$$

Motion compatibility:

$$w_v = \exp\left(-\frac{\|u_i - \hat{u}_j\|^2}{2\sigma_u^2}\right).$$

Material/roughness compatibility:

$$w_\rho = \exp\left(-\frac{|\rho_i - \rho_j|^2}{2\sigma_\rho^2}\right).$$

Flow residual compatibility:

$$w_r = \exp\left(-\frac{r_j^2}{2\sigma_r^2}\right).$$

Sample-quality compatibility:

$$w_q = \mathrm{sat}(q_i).$$

These gates are the anti-ghosting core. A prototype may begin with only w_x, w_d, w_{id}, then add normals, motion, and residuals as the scenes become harder.

7.4 Assignment

For each observation, find the best compatible Gaussian:

$$j^* = \arg\max_j W_{ij}.$$

If $W_{ij^*} < \epsilon_W$, spawn a new Gaussian. Otherwise update G_{j^*}. The CUDA prototype should use a hashed 3D grid plus primitive buckets, fixed-radius cell neighborhoods, and bounded candidate lists. A scalar CPU path may exist only as a tiny correctness oracle; it is not the implementation target.

7.5 State Update

Let α_{ij} be the update weight:

$$\alpha_{ij} = \frac{W_{ij}}{m_j + W_{ij} + \epsilon}.$$

Mean update:

$$\mu_j' = (1 - \alpha_{ij})\mu_j + \alpha_{ij} x_i.$$

Color update:

$$c_j' = (1 - \alpha_{ij})c_j + \alpha_{ij} c_i.$$

Radiance level update:

$$(L_j^k)' = (1-\alpha_{ij})L_j^k + \alpha_{ij}L_i^k.$$

Velocity update:

$$\dot{\mu}_j' = (1-\alpha_{ij})\dot{\mu}_j + \alpha_{ij}\frac{x_i - \mu_j^{prev}}{\Delta t}.$$

Sample mass and age:

$$m_j' = \lambda_m m_j + W_{ij}, \qquad \tau_j' = t.$$

Confidence combines mass, age, variance, and residual consistency:

$$\nu_j = \mathrm{sat}\left(\frac{m_j}{m_j+m_0}\exp(-\lambda_\tau(t-\tau_j))\frac{1}{1+v_j}\exp(-r_j^2/2\sigma_r^2)\right).$$

Variance can be updated with an exponential moment:

$$v_j' = (1-\alpha_{ij})v_j + \alpha_{ij}\|c_i - c_j\|^2.$$

7.6 Covariance Update

For a surface-attached Gaussian, use a tangent-frame covariance:

$$\Sigma_j = R_j \begin{bmatrix} \sigma_{t1}^2 & 0 & 0 \\ 0 & \sigma_{t2}^2 & 0 \\ 0 & 0 & \sigma_n^2 \end{bmatrix} R_j^T.$$

The normal-axis variance should stay small:

$$\sigma_n^2 = \min(\sigma_n^2, \epsilon_n^2).$$

For temporal super-resolution, tangent variances should track projected pixel footprint and sample density:

$$\sigma_t^2 = \max(\sigma_{min}^2, \beta_f A_{pixel\to world}/(m_j+\epsilon)).$$

This makes Gaussians shrink as evidence accumulates but prevents unstable point-like splats.

7.7 Projection to Pixels

Project each Gaussian into camera C_t:

$$p_j = \Pi_t(\mu_j).$$

Approximate the screen-space covariance with the projection Jacobian J_j:

$$\Sigma_{screen,j} = J_j\Sigma_j J_j^T + \Sigma_{pixel}.$$

Pixel contribution:

$$a_{pj} = \nu_j \exp\left(-\frac{1}{2}(p-p_j)^T\Sigma_{screen,j}^{-1}(p-p_j)\right).$$

Visibility/depth testing gates a_{pj} before blending. The first CUDA prototype can use per-tile depth gates and bounded front-to-back candidate lists; later versions can use sorted or weighted visibility like ordinary 3DGS.

7.8 TAA Resolve

The Gaussian history color for pixel p is:

$$H_t(p) = \frac{\sum_j a_{pj} c_j}{\sum_j a_{pj} + \epsilon}.$$

Blend with the current seed color $C_t(p)$:

$$O_t(p) = (1 - \beta_p) C_t(p) + \beta_p H_t(p).$$

Where:

$$\beta_p = \mathrm{sat}\left(\beta_0 \frac{\sum_j a_{pj}}{\sum_j a_{pj} + a_0} \frac{1}{1 + \bar{v}_p} \right).$$

This is the direct replacement for image-space history. It should fail closed: if confidence is low, output the current seed.

7.9 Temporal Super-Resolution Resolve

For output pixel P at high resolution and scale factor s, use the same projection equation but evaluate at the high-resolution pixel center:

$$O_t^{SR}(P) = \frac{\sum_j a_{Pj} c_j + \lambda C_t^{up}(P)}{\sum_j a_{Pj} + \lambda}.$$

C_t^{up} is a simple upsampled current frame. λ prevents stale history from dominating disoccluded regions. This is the first test of integration: the TAA and SR resolves must use the same G_j, not two separate caches.

For 2x and 4x experiments, the only scale-specific term should be the pixel footprint:

$$\Sigma_{pixel}^{SR}(s) = \frac{1}{s^2} \Sigma_{pixel}.$$

If the method needs a different Gaussian state for 2x and 4x, it fails the integration test. A single state may still expose different projection footprints.

7.10 Radiance Cache Query

At a shading point x with normal n, outgoing direction ω_o, roughness ρ, and path-depth level k, query:

$$\hat{L}^k(x, \omega_o) = \frac{\sum_j W_j(x, n, \omega_o, \rho, k) L_j^k}{\sum_j W_j(x, n, \omega_o, \rho, k) + \epsilon}.$$

The query weight reuses the same gates as TAA plus roughness/path-depth compatibility. The first experiment should query only $k = 1$ diffuse or rough indirect radiance. Direct lighting remains sampled normally.

Radiance cache update uses the same assignment but separate moments per path level:

$$(M_{L,j}^k)' = (1-\alpha_{ij})M_{L,j}^k + \alpha_{ij}L_i^k,$$

$$(V_{L,j}^k)' = (1-\alpha_{ij})V_{L,j}^k + \alpha_{ij}\|L_i^k - M_{L,j}^k\|^2.$$

When used as a biased indirect-light estimate, the cache contributes directly. When used as a safer denoising feature, it is projected into a feature frame:

$$F_L(p,k) = \frac{\sum_j a_{pj}L_j^k}{\sum_j a_{pj} + \epsilon}.$$

The unbiased/control-variate path is a later experiment:

$$\hat{I} = G_\theta + \frac{1}{N}\sum_i \frac{f(x_i) - g_\theta(x_i)}{p(x_i)}.$$

The first CUDA prototype should not start here. First prove that the biased cache reduces variance without unacceptable light leaks.

7.11 Path-Space Gaussian Reconstruction

A full path-space Gaussian would live in too many dimensions:

$$\xi = (x_0, \omega_0, x_1, \omega_1, \ldots, x_\ell, \omega_\ell).$$

That is not the right first representation for vkGSplat. Instead, use surface-attached Gaussians as addresses and attach low-dimensional path subslots:

$$A_j^{path}(k,b) = (M_{jkb}, Q_{jkb}, N_{jkb}, R_{jkb}),$$

where k is path depth, b is a coarse directional/material lobe, M is the mean contribution, Q is the second moment, N is sample count, and R is a robust residual. A path event chooses a lobe:

$$b_i = B(\omega_i, \omega_o, \rho_i, material_i).$$

The updated contribution is:

$$Y_i = \frac{\beta_i f_i L_i}{p_i},$$

with throughput β_i, scattering term f_i, incident radiance L_i, and sampling density p_i. Use robust clamping only for the biased reconstruction mode:

$$\bar{Y}_i = \min(\|Y_i\|, c_\sigma\sqrt{Q_{jkb}})\frac{Y_i}{\|Y_i\| + \epsilon}.$$

Then update the lobe moments:

$$N'_{jkb} = N_{jkb} + w_i, \qquad M'_{jkb} = M_{jkb} + \frac{w_i}{N'_{jkb}}(\bar{Y}_i - M_{jkb}),$$

$$Q'_{jkb} = Q_{jkb} + \frac{w_i}{N'_{jkb}}(\|\bar{Y}_i\|^2 - Q_{jkb}).$$

This is implementable as a CUDA `UpdatePathSubslots` kernel. The first test should use a single diffuse lobe so $b = 0$; if that cannot reduce variance without leaking across primitive boundaries, higher-dimensional path-space reconstruction is unlikely to rescue the idea.

7.12 Gaussian Control Variates

There are two estimator modes, and they should be kept separate in code.

Biased reconstruction mode uses the Gaussian cache as a denoised estimate:

$$O_t(p) = C_t^{direct}(p) + \hat{L}_G^{indirect}(p).$$

This is the fastest useful prototype and should be judged by MSE, temporal stability, and light-leak masks.

Control-variate mode treats the Gaussian cache as a predictor:

$$\hat{I}_{CV} = \frac{1}{N} \sum_{i=1}^{N} (Y_i - g_G(\xi_i)) + \hat{G}.$$

Here $g_G(\xi_i)$ is the Gaussian prediction evaluated at the same path sample and $\hat{G}$ is the cache's estimate of the corresponding integral. This is only unbiased if $\hat{G}$ matches the same measure; otherwise it is a biased residual estimator. The CUDA-facing API should therefore expose:

```
RadianceEstimate query_biased_radiance(const ShadingQuery& q) const;
RadianceEstimate query_control_variate(const PathEvent& e) const;
```

The falsifiable experiment is straightforward: compare equal-spp variance and bias against a high-spp reference for three modes: no cache, biased Gaussian cache, and Gaussian residual/control-variate mode. The control-variate path is worth pursuing only if variance drops faster than bias rises.

7.13 Gaussian-Guided Sampling

Once query quality is established, the cache can guide future samples. Use a mixture proposal:

$$p'(y|x) = (1 - \eta) p_{base}(y|x) + \eta p_G(y|x).$$

The contribution remains MIS-weighted:

$$Y_i' = \frac{f(y_i)}{p'(y_i|x_i)}.$$

For the first vkGSplat prototype, p_G should be very simple: choose one of the top-K nearby radiance-bearing Gaussians and sample a cosine lobe around its mean incident direction. The experiment is a kill test. If Gaussian-guided sampling helps only when the scene is already easy, the cache is useful as a denoising feature but not as a path-space variance-reduction engine.

7.14 Motion-Accurate Interpolation

For midpoint synthesis at $\alpha \in (0, 1)$:

$$\mu_j^\alpha = (1-\alpha)\mu_j^0 + \alpha\mu_j^1.$$

Constant-velocity fallback:

$$\mu_j^\alpha = \mu_j^0 + \alpha\Delta t\,\dot{\mu}_j^0.$$

Confidence is reduced by forward/backward disagreement:

$$\nu_j^\alpha = \nu_j^0\nu_j^1 \exp\left(-\frac{\|\Pi_1(\mu_j^1) - (\Pi_0(\mu_j^0) + u_{0\to1})\|^2}{2\sigma_u^2}\right).$$

Splat G_j^α into the midpoint camera and compare against a true rendered middle frame. This is motion-accurate only if silhouettes and depth edges land correctly, not merely if the image looks plausible.

For rigid objects, the preferred update is not learned velocity but engine transform:

$$\mu_j^\alpha = T_{obj}(\alpha)T_{obj}(0)^{-1}\mu_j^0.$$

For non-linear motion, use a cubic trajectory once four frames are available:

$$\mu_j(t) = a_j t^3 + b_j t^2 + c_j t + d_j.$$

Motion confidence is the minimum of visibility confidence, transform confidence, and flow-residual confidence. This ensures frame interpolation fails closed in disoccluded or mismatched regions.

7.15 Frame Generation and Multi-Frame Generation

Motion-accurate interpolation generates one or more intermediate frames between two real frames:

$$\alpha_k = \frac{k}{K+1}, \qquad k = 1, \ldots, K.$$

For each α_k, evaluate $G_j^{\alpha_k}$ and splat with the same visibility/confidence gates used by midpoint interpolation. The frame is not accepted as a confident generated frame unless the per-pixel support is strong:

$$S_\alpha(p) = \sum_{j\to p} a_{pj}^\alpha \nu_j^\alpha, \qquad S_\alpha(p) > S_{min}.$$

Pixels with weak support become holes:

$$H_\alpha(p) = \begin{cases} 1, & S_\alpha(p) \le S_{min} \text{ or } D_\alpha(p) = 1, \\ 0, & \text{otherwise.} \end{cases}$$

The generated output is therefore explicitly decomposed:

$$O_\alpha(p) = (1 - H_\alpha(p))O_\alpha^{gauss}(p) + H_\alpha(p)O_\alpha^{fallback}(p).$$

For a non-neural prototype, $O_\alpha^{fallback}$ is nearest real-frame hold or image-space reprojection. For a neural prototype, it is a bounded residual resolver conditioned on the hole mask. This is important: vkGSplat should not pretend 3DGS solved disoccluded background generation. It should make unknown regions first-class.

For extrapolative frame generation, predict from one real frame:

$$\hat{\mu}_j(t+\Delta) = \mu_j(t) + \Delta\dot{\mu}_j(t) + \frac{1}{2}\Delta^2\ddot{\mu}_j(t).$$

Uncertainty must grow with prediction horizon:

$$\hat{\Sigma}_j(t+\Delta) = \Sigma_j(t) + \Delta^2\Sigma_{\dot{\mu},j} + \Delta^4\Sigma_{\ddot{\mu},j} + Q_j\Delta,$$

$$\hat{\nu}_j(t+\Delta) = \nu_j(t)\exp(-\Delta/\tau_{motion})\exp(-r_j^2/2\sigma_r^2).$$

This turns long-horizon frame generation into a confidence problem rather than a hallucination problem. The first experiment should generate only one midpoint. The second can generate $K = 2$ intermediate frames. Extrapolation should wait until interpolation passes the silhouette and disocclusion tests.

7.16 Frame-Generation Experiments

Use three clips:

1. **Rigid transform clip:** one object with known object transform. Expected pass: generated silhouettes match the held-out real frames.
2. **Camera strafe clip:** static scene with disocclusion. Expected pass: disoccluded holes are masked and do not receive high confidence.
3. **Specular/deforming clip:** intentionally hard case. Expected pass: residual and confidence metrics predict failure rather than hiding it.

Report generated-frame metrics separately from TAA/SR metrics:

$$E_{FG} = E_{mid} + \lambda_s E_{sil} + \lambda_h E_{hole} + \lambda_f E_{flow}.$$

The shared-state thesis is supported only if frame generation reuses the same Gaussian core and if its failure masks are useful to the TAA/SR/neural paths.

7.17 Neural Feature Projection

The optional neural resolver should not receive an opaque "Gaussian image." It should receive explicit feature planes:

$$\Phi_t(p) = [H_t(p), F_L(p,0), \ldots, F_L(p,K), \bar{v}_p, \bar{\nu}_p, \bar{\tau}_p, \bar{r}_p, D_p, M_p].$$

Where D_p is a disocclusion likelihood and M_p is a motion-consistency feature. A compact model can then learn the residual:

$$O_t(p) = O_t^{analytic}(p) + N_\theta(C_t, depth_t, velocity_t, \Phi_t)(p).$$

The key ablation is equal model size with and without Φ_t. If Gaussian features do not improve quality or reduce model size, the integration layer is less useful than hoped.

7.18 Split, Merge, and Prune

Split a Gaussian when it is high-confidence but high-variance:

$$m_j > m_{split} \quad \text{and} \quad v_j > v_{split}.$$

Merge two Gaussians when they are compatible and neither split nor depth gates object:

$$D_{KL}(\mathcal{N}_a, \mathcal{N}_b) < \epsilon_{merge}, \quad id_a = id_b.$$

Prune stale Gaussians:

$$\nu_j < \epsilon_\nu \quad \text{or} \quad t - \tau_j > T_{max}.$$

The first CUDA implementation can skip split/merge and only spawn/prune. Split and merge become necessary if the shared state blurs thin geometry or grows without bound.

7.19 Visibility and Lifetime Update

Visibility is the part most likely to decide whether the unified state works. Every frame, a Gaussian can be observed, occluded, disoccluded, or simply outside the frustum. Treat these as different lifetime events:

$$\ell_j^t \in \{\text{visible}, \text{occluded}, \text{disoccluded}, \text{out_of_frustum}\}.$$

If a projected Gaussian lands behind the current depth buffer by more than a threshold, it is occluded, not stale:

$$\ell_j^t = \text{occluded} \quad \text{if} \quad d_j^t - D_t(p_j) > \epsilon_{occ}.$$

If it lands in front of current depth or disagrees with primitive ID, it is potentially stale:

$$\ell_j^t = \text{disoccluded} \quad \text{if} \quad D_t(p_j) - d_j^t > \epsilon_{front} \quad \text{or} \quad id_j \neq ID_t(p_j).$$

Use different confidence decay rates:

$$\nu_j^{t+1} = \begin{cases} \min(1, \nu_j^t + \Delta_\nu), & \ell_j^t = \text{visible}, \\ \lambda_{occ}\nu_j^t, & \ell_j^t = \text{occluded}, \\ \lambda_{dis}\nu_j^t, & \ell_j^t = \text{disoccluded}, \\ \lambda_{frustum}\nu_j^t, & \ell_j^t = \text{out_of_frustum}. \end{cases}$$

Require $\lambda_{dis} < \lambda_{occ}$. Occluded objects should be allowed to survive; disoccluded stale history should disappear quickly. This distinction is essential for frame interpolation and TAA: otherwise the cache either ghosts or forgets too aggressively.

7.20 Tensorized Kernel ABI and M–N–K Estimates

The intended vkGSplat kernel ABI is tensorized, even when the implementation is custom CUDA rather than generic PyTorch. The important boundary is:

All persistent state, all seed inputs, and all reconstruction outputs should be represented as contiguous tensor-shaped structure-of-arrays buffers. Irregular operations such as compaction, grid build, assignment, split/merge, and splat resolve remain custom CUDA tensor operators.

This gives vkGSplat two front doors over the same kernels. The research front door accepts `torch.Tensor`/NVDIFFRAST buffers; the runtime front door accepts Vulkan-exported CUDA memory with the same shapes and strides. PyTorch should be a caller and allocator for experiments, not the owner of the renderer architecture.

For the rest of this note, use the following M–N–K convention:

- M is the number of observations entering a kernel: raster pixels, depth-peel pixels, stochastic samples, or compacted seed hits.
- N is the number of active persistent Gaussian records available to query, update, project, or resolve.
- K is the feature width carried by each observation, Gaussian, or projected feature plane. Local nearest-neighbor candidate count is written as C so it is not confused with feature width.

The acquisition tensor contract should start as:

$$\begin{aligned}
R &\in \mathbb{R}^{B\times H\times W\times 4}, && R = (u, v, z/w, id_{tri} + 1),\\
C_{img} &\in \mathbb{R}^{B\times H\times W\times 4}, && \text{radiance or color plus confidence/alpha},\\
X &\in \mathbb{R}^{B\times H\times W\times 3}, && \text{world position from interpolation or ray hit},\\
Nrm &\in \mathbb{R}^{B\times H\times W\times 3}, && \text{world normal},\\
V_{px} &\in \mathbb{R}^{B\times H\times W\times 2}, && \text{previous-pixel minus current-pixel motion}.
\end{aligned}$$

A tensorized extraction kernel compacts valid hits into structure-of-arrays:

$$\begin{aligned}
X_s &\in \mathbb{R}^{M_s\times 3}, \quad N_s \in \mathbb{R}^{M_s\times 3}, \quad L_s \in \mathbb{R}^{M_s\times 4},\\
p_s &\in \mathbb{R}^{M_s\times 2}, \quad v_s \in \mathbb{R}^{M_s\times 2}, \quad b_s \in \mathbb{R}^{M_s\times 2},\\
q_s &\in \mathbb{R}^{M_s\times 2}, \quad I_s \in \mathbb{Z}^{M_s\times 4}.
\end{aligned}$$

Here $q_s = (z/w, \nu)$ and $I_s = (id_{tri}, id_{px}, id_{batch}, flags)$. The current minimal sample width is therefore

$$K_s = 18 \text{ fp32 lanes} + 4 \text{ int32 lanes} \approx 88 \text{ bytes per compact sample.}$$

Material IDs, roughness/metalness, per-sample variance, depth derivatives, and path-space throughput should be appended as extra tensor columns. The first CUDA implementation should keep the minimal K_s above, then measure which extra channels actually improve denoising, temporal stability, or frame interpolation.

The persistent Gaussian state should also be SoA tensors:

$$\begin{aligned}
\mu &\in \mathbb{R}^{N\times 3}, && \Sigma \in \mathbb{R}^{N\times 6}, && \bar{L} \in \mathbb{R}^{N\times 4},\\
\sigma_L^2 &\in \mathbb{R}^{N\times 4}, && \bar{n} \in \mathbb{R}^{N\times 3}, && \bar{v} \in \mathbb{R}^{N\times 3},\\
h &\in \mathbb{R}^{N\times K_h}, && meta \in \mathbb{Z}^{N\times K_i}.
\end{aligned}$$

A no-neural analytic state needs roughly $K_g = 32$–48 fp32 lanes plus $K_i = 4$–8 integer lanes per Gaussian: mean, covariance, radiance moments, normal, velocity, mass/confidence, timestamps, primitive/material identity, cell key, and flags. At fp32 this is about 160–240 bytes per Gaussian. A small learned residual can add $K_h = 16$–64 feature lanes, increasing the state by 64–256 bytes per Gaussian. This is why the default experiment should prove value with $K_h = 0$ before adding neural features.

Let $P = HW$, S be the number of acquisition samples per pixel, L the number of depth layers, and ρ the valid-surface hit ratio. Then

$$M_{raw} = BPSL, \qquad M_s \approx \rho M_{raw}.$$

For DLSS-style super-resolution the output pixel count grows with the linear scale r:

$$P_{out} = r^2 P,$$

but M_s does not need to grow with r unless vkGSplat deliberately acquires extra subpixel samples. This distinction is important: the Gaussian state is supposed to amortize history and geometry so a 2× or 4× resolve can increase projected output work without requiring 4× or 16× new shading work every frame.

The first-order M and N budgets are:

Mode	P	M_s single layer	M_s 4 jitter/depth	N retained	State memory
1080p	2.07M	1.2–2.0M	5–8M	1.5–4M	0.25–1.0GB
1440p	3.69M	2.2–3.5M	9–14M	3–7M	0.5–1.7GB
4K	8.29M	5–8M	20–32M	6–16M	1.0–4.0GB

The N range assumes that visible surfaces need roughly 0.5–1.5 Gaussians per currently valid pixel after merging, plus a history retention factor of 1.5–3.0 for temporarily occluded or subpixel evidence. Thin geometry, particles, foliage, transparencies, and depth peeling push N upward; flat static walls should merge down aggressively. If N must exceed about $2P$ for ordinary opaque scenes, the covariance/update rules are probably too fragmented. If $N < 0.25P$ cannot preserve edges and subpixel motion, the state is probably too coarse for DLSS-class reconstruction.

The key kernel shapes are:

Kernel	Shape	Main cost	Health target
Extract hits	$M_{raw} \to M_s$	$O(M_{raw})$	bandwidth-bound, one pass
Build grid	$N \to$ cells	$O(N)$ + scan	B_{p99} bounded
Assign samples	$M_s \times C$	local search	$C \leq 32$ typical, $C \leq 64$ p99
Reduce updates	$M_s \to N$	sort or segmented reduce	fewer atomics than direct update
Project splats	$N \to$ tiles	tile-list emission	reused by TAA/SR/framegen
Resolve	$P_{out} \times K_p$	tile traversal	no duplicated feature passes

Projected resolver feature width should begin small:

$$K_p \approx 16\text{–}32 \quad \text{for} \quad (L, \alpha, z, n, v, \nu, \sigma^2, id, coverage, age, flags).$$

where integer IDs can live in parallel integer tensors. A neural resolver may consume $K_p = 32$–96 channels, but the pass/fail experiment should compare against the analytic $K_p \leq 32$ case first.

These estimates imply three CUDA milestones:

1. **M0 tensor extraction:** $M_s \leq 2\text{M}$, $N = 0$, $K_s = 18 + 4$. Demonstrate NVDIFFRAST/Vulkan seed tensors can become compact sample tensors in less than one millisecond at 1080p-class sizes on a desktop CUDA GPU.

2. **M1 persistent update:** $M_s = 1$–4M, $N = 1$–3M, $K_g \leq 48$. Demonstrate bounded grid occupancy, stable assignment, and lower temporal variance than image-space history on camera motion.

3. **M2 DLSS-class resolve prototype:** $M_s = 2$–8M, $N = 3$–8M, $K_p = 16$–32 analytic or $K_p \leq 96$ with a small neural residual. Demonstrate TAA/SR/frame-interpolation benefit from one shared state rather than separate feature-specific histories.

A negative result is useful if it is dimensional: too much M bandwidth, too large an N retention factor, or too wide a K for the quality gained. The experiments should report all three numbers for every image-quality result.

7.21 Mixed-Precision Gaussian State and FP16/FP8/FP4 Policy

vkGSplat should use lower precision aggressively, but only with a two-copy rule: keep the canonical estimator state accurate enough to remain stable, and materialize lower-precision tile packets for bandwidth- and tensor-core-heavy work. This is different from quantizing the whole renderer. Geometry, motion, visibility, and estimator control should not become FP4-only state.

The hardware reason is strong. Hopper-class Tensor Cores expose FP8 paths with E4M3 and E5M2 formats, where E4M3 gives more precision and E5M2 gives wider range [31, 32]. CUDA also exposes FP4, FP6, FP8, FP16, and BF16 math/intrinsic support in the low-precision API surface [33]. Blackwell-style NVFP4 uses an E2M1 element with hierarchical block scaling, making it a plausible tensor format for neural weights, activations, and compact learned features, but not a safe standalone geometry format [34]. The OCP MX specification is also standardizing microscaled FP4/FP6/FP8-style interchange formats, which is relevant if vkGSplat later targets non-CUDA accelerators [35].

The precision policy should be field-specific:

State or packet field	Preferred format	Rationale
Camera matrices, canonical centers μ, covariance update accumulators	FP32	Keeps reprojection, assignment, and long-lived estimator state stable.
Canonical covariance factors, normal, velocity, opacity, radiance moments	FP32/FP16	Store master accumulators in FP32 where drift matters; expose FP16 render views.
Per-tile projected features, analytic resolver features, radiance-cache feature vectors	FP16/FP8	Bandwidth-heavy and tensor-friendly; use per-tile or per-block scaling.
Neural resolver weights, latent residual channels, compressed cache codes	FP8/FP4	Best FP4 target; require calibration or quantization-aware training.
Primitive IDs, material IDs, tile indices, flags, sample counts, timestamps	integer	Never float-quantize identity or ordering metadata.
Variance, confidence, visibility gates, disocclusion scores	FP16/FP32	These control history acceptance; quantization error becomes ghosting.

The implementation should therefore split persistent state from working state:

$$S_j^{master} = (\mu_j, \Sigma_j, \bar{L}_j, \bar{n}_j, \bar{v}_j, \nu_j, meta_j) \quad \rightarrow \quad P_{j,t}^{tile} = Q_{p,t}(S_j^{master}),$$

The master copy uses FP32/FP16 plus integer metadata. The tile packet is a transient low-precision view used by projection, feature resolve, radiance-cache lookup, or neural decode. For a block b of feature values x_i, a generic block-scaled packet is

$$q_i = Q_p(x_i/s_b), \qquad \tilde{x}_i = s_b D_p(q_i),$$

where $p \in \{\text{fp8}, \text{fp4}\}$, Q_p quantizes, D_p dequantizes, and s_b is a tile, cache-page, or micro-block scale. NVFP4-style packets add a tensor-level scale:

$$\tilde{x}_i = s_{global} s_{block} D_{E2M1}(q_i).$$

This is a natural match for Gaussian radiance cache pages: the page table can store both residency metadata and quantization scales, just as KV-cache systems store block/page metadata for attention. The page size should be chosen so one cache page holds a small integer number of Gaussian feature blocks and their scales, for example 16–64 Gaussians times K_h = 16–64 feature lanes.

Concrete CUDA kernels:

1. **PackGaussianTilePackets:** read canonical SoA state, project or select tile-local Gaussians, compute per-block scales, and emit FP16, FP8, or FP4 packet tensors plus scales.

2. **TensorFeatureProjectFP8:** evaluate small feature projections or MLP-style basis transforms with FP8 inputs and FP16/FP32 accumulation.

3. **QuantizedRadianceCacheLookup:** gather page-resident radiance features, dequantize into registers/shared memory, and blend with analytic visibility and confidence gates.

4. **NeuralResolveFP4FP8:** run the optional transformer/MLP resolver on FP8 activations and FP4/FP8 weights, accumulating in FP16 or FP32.

5. **ApplyGaussianDeltas:** write updates back to the canonical state in FP32/FP16 only after compatibility gates and reductions.

The first experiments should treat precision as an ablation axis rather than a one-way optimization:

Ablation	Change	Pass condition
P0	FP32 canonical state and FP32/FP16 resolve baseline	Reference.
P1	FP16 render view for color, opacity, normal, velocity, and covariance factors	$\leq$ 0.2dB PSNR loss and no visible increase in flicker.
P2	FP8 projected features and radiance-cache feature pages	$\leq$ 0.3dB loss, $\leq$ 10% flicker increase, measurable bandwidth reduction.
P3	FP4 latent resolver weights or residual channels only	$\leq$ 0.5dB loss versus FP8 resolver and at least 1.3$\times$ resolver throughput or bandwidth gain.
P4	FP4 geometry/covariance negative control	Expected to fail; confirms why canonical geometry remains higher precision.

Report image metrics (PSNR, SSIM, LPIPS), temporal metrics (flicker, disocclusion, interpolation endpoint error), and GPU metrics (cache bandwidth, tensor-core utilization, packet conversion cost, and memory footprint). A successful result would show that FP16/FP8 reduce the cost of the shared Gaussian state without weakening history rejection. FP4 should only become useful once the work has been turned into a calibrated tensor decode problem.

7.22 Transformer Architecture: Optional Resolver, Not Core State

Transformer architecture is relevant, but it should enter vkGSplat at the resolver boundary rather than at the core Gaussian-state boundary. NVIDIA's DLSS 4 moved Super Resolution, Ray Reconstruction, and DLAA from a CNN lineage to a real-time vision-transformer model with self-attention over frame and multi-frame evidence [29]. NVIDIA's DLSS 4.5 announcement then describes a second-generation transformer model for Super Resolution [27, 28]. This is strong evidence that a DLSS-class product should expect transformer-style neural resolving to be competitive.

For vkGSplat, however, a transformer should not replace the explicit Gaussian state. The project hypothesis is that geometry, visibility, radiance, motion, and confidence should be maintained explicitly in CUDA tensors. A transformer, if used, consumes the projected Gaussian feature tensors and predicts a bounded residual:

$$\hat{I}_{hi} = R_\theta(F_t, F_{t-1}, D_t, V_t, M_t) + I_{analytic},$$

where $F_t \in \mathbb{R}^{B \times H_o \times W_o \times K_p}$ is the projected Gaussian feature tensor, D_t is depth, V_t is motion, M_t is a mask/ID feature tensor, and $I_{analytic}$ is the non-neural Gaussian/TAA/SR resolve. The model should learn what the analytic state cannot model cleanly: high-frequency shading residuals, difficult disocclusions, specular flicker, and perceptual anti-aliasing. It should not be responsible for discovering basic geometry or motion from scratch.

Operational placement. The transformer is used only in the decode resolver, after the Gaussian state has already been updated and projected. It is not part of the prefill renderer, not part of Gaussian assignment/update, and not part of the visibility or radiance-cache acceptance gates. Those stages must remain inspectable CUDA kernels so the no-transformer baseline is always valid.

Pipeline stage	Transformer?	Reason
Prefill/acquisition	No	Produce factual seed buffers: color, depth, IDs, normals, motion, and variance.
Gaussian update/cache	No	Maintain geometry, visibility, radiance, confidence, and lifetime as explicit state.
Feature projection	No model	Project Gaussian and engine features into tile/window tensors for the resolver.
Decode resolver	Optional yes	Predict bounded residuals, confidence repair, SR detail, or disocclusion fill.
Final composite	No	Apply residuals and masks to the analytic resolve with clamps and debug counters.

In kernel terms, the optional path is:

$$\texttt{ProjectFeatures} \rightarrow \texttt{PackResolverTokens} \rightarrow \texttt{RunTransformerResolver} \rightarrow \texttt{CompositeResolverOutput}.$$

The token tensor can be written as

$$Z_t = \mathrm{PackTokens}(F_t, A_t, H_{t-k:t-1}),$$

where A_t is the current seed bundle and $H_{t-k:t-1}$ are motion-aligned history features. The resolver predicts bounded corrections:

$$(\Delta I_t, \Delta c_t, \Delta m_t) = T_\theta(Z_t), \qquad \hat{I}_t = I_{analytic} + \mathrm{clamp}(\Delta I_t).$$

Here Δc_t can adjust confidence and Δm_t can repair masks or holes. The clamps are not cosmetic: they enforce that the transformer is a bounded decoder over explicit state, not the owner of geometry or temporal truth. Disabling the transformer should reduce to the analytic CUDA resolve without changing the prefill, Gaussian state, cache layout, or metrics.

A practical transformer budget should be windowed and temporal-local rather than global full-frame attention. Full attention over P pixels scales as $O(P^2)$ and is not a credible real-time starting point. Window attention with window size $w \times w$ scales as

$$O(P\, w^2\, K_p),$$

plus small cross-frame token mixing. For 1080p, $w = 8$ gives 64-token local attention; $w = 16$ gives 256 tokens and should be treated as a high-quality ablation, not the default. The first neural resolver should therefore be:

- a tiny windowed transformer or hybrid Conv/Transformer block,
- input $K_p = 32$–96 projected Gaussian and engine features,
- two to four previous-frame feature tensors at most,
- output either residual color, confidence/mask correction, or SR color,
- trained and evaluated against an equal-parameter image-space model.

The transformer decision should be staged:

1. **No-transformer baseline:** analytic CUDA tensors only. This must already demonstrate that shared Gaussian state improves at least one of TAA, SR, denoising, radiance caching, or interpolation.

2. **Small transformer resolver:** consume projected Gaussian feature tensors and compare against an image-only transformer with the same parameter count and input resolution.

3. **Product-scale transformer:** only after the shared state wins the ablation. This path may resemble DLSS-class neural resolving, but it is not needed to validate the vkGSplat integration hypothesis.

The pass/fail rule is simple. If a transformer improves both image-space and Gaussian-feature versions equally, the value is mostly neural resolving. If the same transformer is smaller, faster, or more stable when fed projected Gaussian features, then the explicit Gaussian state is helping. Record M, N, K_p, parameter count, attention window size, and frame history length for every neural experiment.

7.23 Why Transformer/LLM Architecture Reuse Is Plausible

The striking architectural point is that vkGSplat can look like graphics at the API boundary while looking like Transformer serving inside the expensive part of the machine. The prefill stage acquires structured evidence from the scene: color, depth, IDs, barycentrics, normals, motion, visibility, and sparse radiance samples. The decode stage repeatedly queries persistent state, pages Gaussian/radiance features, projects local tokens, and resolves pixels, higher resolution samples, or intermediate frames. That is much closer to a stateful cache-plus-tensor workload than to a classical fixed-function graphics pipeline.

With no learned model, the analogy is still useful because Gaussian history and radiance cache pages behave like a structured memory. With an optional transformer resolver, the analogy becomes literal: projected Gaussian features, tile features, and motion-aligned history become tokens, while the Gaussian radiance cache plays the role of a geometry-aware KV cache. This does not mean vkGSplat is an LLM. It means the hardware primitives that make LLM serving fast – tiled matrix engines, low-precision formats, cache paging, block tables, gather/scatter, reductions, and bandwidth scheduling – are also the right primitives for the decode half of this renderer.

Transformer/LLM serving idea	vkGSplat counterpart	Hardware implication
Prompt prefill	Raster/ray/splat acquisition produces seed evidence A_t	Keep graphics-specific logic narrow and evidence-oriented.
KV cache	Persistent Gaussian history, radiance cache, motion state, and feature pages	Invest in SRAM/HBM bandwidth, paging, and cache residency policy.
Token blocks/pages	Tile pages, Gaussian pages, radiance-cache pages, and feature blocks	Reuse block tables, page allocators, and compaction/eviction logic.
Attention over tokens	Local attention or analytic mixing over pixels, Gaussians, tiles, and history frames	Favor block-sparse tensor kernels and tile-local reuse.
Decode loop	Per-frame TAA/SR/denoise/framegen resolve from persistent state	Optimize repeated cache queries and low-launch-count decode kernels.
Quantized KV cache	FP8/FP4 Gaussian feature pages and neural resolver packets	Reuse low-precision scaling, calibration, and packet formats.
Continuous batching	Batched cameras, views, scenes, or training-data jobs	Schedule many small tile/view jobs like serving requests.
Speculative decoding	Motion-predicted intermediate frames or candidate disocclusion fills	Add cheap prediction paths with later validation/rejection.

A useful mental diagram is:

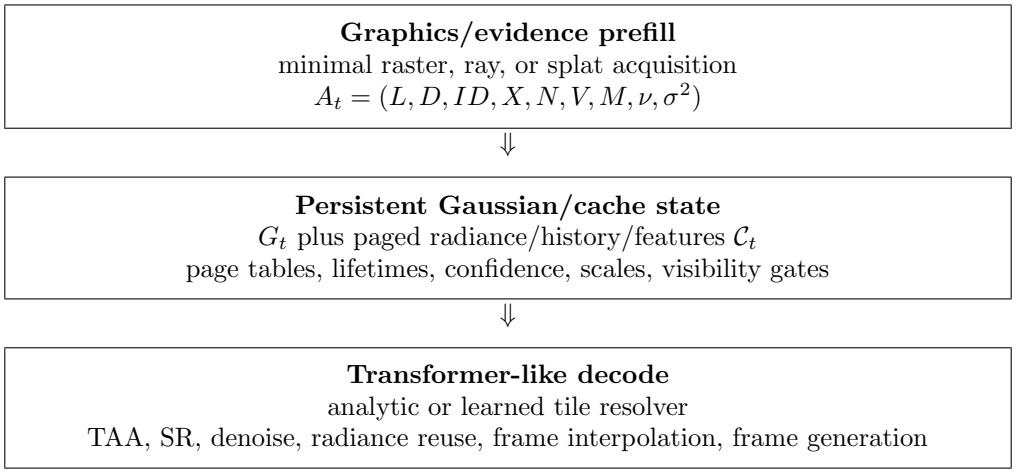

Figure 1: vkGSplat as structured evidence acquisition plus stateful decode. The prefill side benefits from a small amount of graphics-specific visibility hardware. The decode side is dominated by tensor, cache, paging, quantization, and scheduling machinery that overlaps strongly with Transformer serving.

This leads to a concrete hardware rule. If area is scarce, graphics-specific hardware should be added only where it improves evidence quality per joule: depth, primitive IDs, barycentrics, motion vectors, simple texture/material samples, and perhaps ray-hit evidence. The larger investment should remain in the decode substrate: tensor engines, SRAM/HBM bandwidth, page tables, low-precision feature packets, tile scheduling, and sparse gather/reduce kernels. A full legacy raster/ROP/texture/RT stack is less attractive unless experiments show that the prefill evidence

is the dominant quality or energy bottleneck.

The falsifiable experiment is an area-proxy ablation, not just an image-quality ablation. Hold total time or bandwidth budget fixed and compare:

1. **graphics-heavy prefill, weak decode:** higher-spp or higher-res seed buffers, but small Gaussian/cache decode;
2. **minimal prefill, strong decode:** sparse seed buffers, larger Gaussian state, better cache paging, and tensorized decode;
3. **minimal prefill plus transformer resolver:** same seed and state as (2), but add a small learned tile resolver;
4. **balanced bridge:** minimal prefill plus a narrow tile-splat accelerator for Gaussian binning, projection, and compositing.

If (2) or (3) wins at equal bandwidth and latency, vkGSplat should lean toward LLM-style tensor/cache architecture. If (1) wins decisively, then more graphics prefill hardware is justified. If (4) wins, the right hardware is not a full classic graphics pipeline but a narrow Gaussian/tile bridge between acquisition and decode.

7.24 Prefill/Decode Split and Hardware Investment

vkGSplat should explicitly adopt a prefill/decode architecture. The prefill stage is the expensive real-frame evidence stage; the decode stage is the cheaper reconstruction/generation stage that reuses persistent Gaussian and feature caches.

$$\textsc{Prefill}(A_t, G_{t-1}) \to (G_t, \mathcal{C}_t),$$
$$\textsc{Decode}(G_t, \mathcal{C}_t, \alpha, r) \to \hat{I}^{(r)}_{t+\alpha}.$$

Here A_t is the acquired real-frame evidence tensor bundle, G_t is the persistent Gaussian state, $\mathcal{C}_t$ is the tile/feature/cache bundle, α is the generated-frame time offset, and r is the super-resolution scale. The prefill stage performs acquisition, hit extraction, assignment, Gaussian update, visibility decay, grid build, tile-list construction, and optional neural key/value construction. The decode stage projects/querys this state to produce TAA, SR, DLAA, frame interpolation, or generated frames.

The average cost of one real frame plus q decoded outputs is approximately

$$T_{avg}(q) = \frac{T_{prefill} + qT_{decode}}{q+1}.$$

The architecture is useful only if $T_{decode} \ll T_{prefill}$ and cache invalidations remain spatially sparse. A rough cost model is

$$T_{prefill} \sim aM_{raw} + bM_sC + cN + T_{scan/sort} + T_{state},$$
$$T_{decode} \sim dP_{out}K_p + eN_{vis} + T_{attn/cache},$$
$$T_{attn/cache} \sim fP_{out}w^2K_a \quad \text{for windowed attention.}$$

This model clarifies the hardware bet. Rasterization, texture, and ray-tracing hardware remain essential evidence engines. They generate primary visibility, texture/material samples, ray-traced

lighting evidence, shadows, reflections, disocclusion masks, and correctness anchors. Tensor hardware and KV-cache-like memory systems are the likely decode engines: they turn sparse explicit state and projected feature tensors into high-rate frames, super-resolution outputs, and neural residuals.

Therefore vkGSplat should not frame the choice as raster/texture/RT *versus* tensor/KV cache. The better architecture is:

> Rasterization, texture, and ray tracing produce high-quality prefill evidence; tensor cores, cache residency, sparse lookup, and KV-like feature caches make repeated decode cheap.

For vkGSplat research effort, the marginal investment should lean toward the second half because ordinary rasterization already exists while DLSS-class multi-decode reconstruction depends on cache reuse. A practical effort split is:

Area	Suggested emphasis	Reason
Raster/texture/RT acquisition	20–30%	reliable evidence and baselines
Tensorized Gaussian/cache kernels	40–50%	core vkGSplat differentiator
KV-like neural/tile feature cache	15–25%	makes multi-decode economical
Interop/profiling/evaluation	10–15%	proves real frame-time value

For hardware architecture, the recommendation is similar but more balanced: keep improving raster, texture, and ray traversal enough to feed accurate prefill frames, but put the speculative DLSS-5-class upside into tensor throughput, low-latency cache residency, sparse/tiled feature lookup, fast quantized feature storage, and synchronization between graphics and compute. A Gaussian-aware cache is not literally an LLM KV cache, but it plays an analogous role: expensive evidence is encoded once, then many decode queries reuse compact keys, values, confidence, visibility, and motion features.

A concrete cache layout for experiments is:

$$\mathcal{C}_t = (\texttt{tile_ranges}, \texttt{gaussian_indices}, K_t, V_t, M_t),$$

where $K_t \in \mathbb{R}^{T \times B_t \times K_a}$ stores per-tile keys, $V_t \in \mathbb{R}^{T \times B_t \times K_v}$ stores projected Gaussian/radiance values, M_t stores masks, depth ranges, IDs, and confidence, T is the number of tiles, and B_t is a capped per-tile cache budget. Decode then queries only the local tile and neighbor tiles, not the entire Gaussian set.

The pass/fail experiments should be:

1. **Prefill amortization:** compare monolithic update+resolve against prefill once plus 1, 2, 4, and 6 decodes. Report $T_{avg}(q)$, cache memory, ghosting, and disocclusion error.

2. **Cache quantization:** compare fp32, fp16, fp8/int8-like cached features for K_t, V_t at fixed quality.

3. **KV-cache ablation:** decode from raw projected Gaussians, from tile feature caches, and from a small transformer with cached keys and values.

4. **Partial refresh:** invalidate only tiles/primitives with large disocclusion or motion residuals and measure whether quality recovers without full prefill.

The decision rule is: invest more in tensor/KV-cache optimization if prefill amortization gives at least 2×–4× displayed-frame throughput at acceptable temporal error, and if cache memory stays within the N, K budgets above. If decode quality collapses without frequent full prefill, invest more in acquisition and motion/visibility hardware instead.

7.25 Reusing KV-Cache Optimization for Gaussian Radiance Cache

KV-cache hardware and software optimization is directly relevant to vkGSplat, but the reuse is architectural rather than literal. LLM inference caches keys and values for previously processed tokens so repeated decode steps avoid recomputing the prefix. A Gaussian radiance cache should cache spatial, visibility-aware keys and radiance/history values so repeated TAA/SR/framegen decodes avoid re-querying or re-updating the entire Gaussian state.

LLM KV cache	Gaussian radiance cache	Reusable idea
Token position	tile/cell/Gaussian address	logical-to-physical block map
Key vector	position, normal, depth, ID, motion, confidence	query-key scoring
Value vector	radiance, SH/lobe moments, variance, age	cached decode payload
Prefix reuse	static or slow scene regions	sharing across generated frames
Paged blocks	tile/Gaussian pages	low-fragmentation dynamic allocation
KV quantization	radiance/feature quantization	bandwidth and residency reduction
Copy-on-write	page update on visibility/material change	sparse refresh

PagedAttention shows that dynamic KV caches benefit from block paging, reduced fragmentation, and flexible sharing across requests [36]. vAttention argues for retaining contiguous virtual memory while allocating physical memory on demand, avoiding kernel complexity caused by explicitly paged layouts [37]. vkGSplat should test both ideas in Gaussian form: explicit block tables are easier to prototype and inspect, while contiguous virtual addressing may be better if CUDA virtual-memory APIs keep kernels simpler.

A Gaussian radiance page should have a logical address

$$a = (tile, depth_layer, primitive_group, material_group, epoch),$$

and a physical page containing a capped number B_p of cache records:

$$\mathcal{P}_a = (K_a, V_a, M_a, E_a),$$

where

$$\begin{aligned} K_a &\in \mathbb{R}^{B_p \times K_{key}}, \\ V_a &\in \mathbb{R}^{B_p \times K_{val}}, \\ M_a &\in \mathbb{Z}^{B_p \times K_{meta}}, \\ E_a &\in \mathbb{R}^{B_p \times K_{err}}. \end{aligned}$$

The key tensor should include compact geometry and identity features: position relative to tile, depth range, normal cone, primitive/material ID hashes, velocity, confidence, and visibility

age. The value tensor should store radiance and reconstruction payloads: RGB or SH coefficients, indirect-radiance moments, lobe/roughness bins, variance, temporal confidence, and optional neural features. The error tensor stores residual statistics used for eviction, refresh, and quantization health.

The decode query becomes local attention or local compatibility over a small set of pages:

$$\hat{L}(q) = \sum_{a \in \mathcal{N}(tile(q))} \sum_{j \in \mathcal{P}_a} \mathrm{softmax}_j \left(\frac{\phi(q)^\top K_{a,j}}{\sqrt{K_{key}}} - \lambda_d \Delta d - \lambda_n \Delta n - \lambda_{id} \Delta id \right) V_{a,j}.$$

The hard visibility and identity gates remain outside the softmax. This is a major difference from LLM KV cache: wrong reuse in text usually changes token probability smoothly; wrong reuse in rendering creates obvious ghosting, light leaks, and depth-edge errors. Therefore the page table must be geometry-aware:

- **Hard invalidation:** primitive/material mismatch, large depth residual, disocclusion, or topology epoch mismatch invalidates records.
- **Soft decay:** occlusion age, variance, and motion residual lower confidence without immediately freeing the page.
- **Copy-on-write:** static shared pages can serve multiple decode outputs until a prefill update touches their primitive/tile group.
- **Motion prefetch:** tile pages are prefetched along screen-space and Gaussian velocities for frame interpolation.

Quantization should be asymmetric. Geometry, depth bounds, primitive identity, and confidence should stay fp16/fp32 or integer-exact until experiments prove otherwise. Radiance values, SH/lobe features, and neural features are better candidates for fp16, fp8, int8, or product/vector quantization. The right first experiment is not maximum compression; it is measuring when quantized cached radiance begins to cause visible flicker, color bias, or light leakage:

$$Q_{safe} = \min_Q \{\mathrm{bytes}(Q) : \Delta E_{temporal} < \epsilon_t, \Delta E_{edge} < \epsilon_e, \Delta E_{radiance} < \epsilon_L\}.$$

The CUDA kernels implied by this design are:

1. `BuildRadiancePageKeys`: write page keys from active Gaussians, tile lists, depth ranges, and primitive/material groups.
2. `AllocateRadiancePages`: allocate physical pages from a device pool using a block table or virtual-memory-backed allocator.
3. `ScatterRadianceValues`: write radiance, variance, confidence, and optional neural values into page-local SoA tensors.
4. `DecodeRadiancePages`: query local pages during TAA/SR/framegen resolve using hard gates plus local key-value scoring.
5. `InvalidateRadiancePages`: clear or decay pages after disocclusion, primitive change, or large residual error.

6. `QuantizeRadiancePages`: pack stable value pages to fp8/int8-like formats and leave unstable pages at higher precision.

This section turns the hardware question into a concrete vkGSplat thesis:

> The reusable hardware idea is not "attention for everything." It is paged, quantized, high-residency, queryable state for repeated decode. Gaussian radiance cache is the graphics analogue of KV cache, with geometry-aware invalidation added.

The pass/fail experiments are:

1. **Paged vs contiguous cache:** compare explicit page tables, contiguous virtual allocation, and simple flat arrays. Report page waste, allocator time, decode time, and kernel complexity.
2. **Radiance-cache quantization:** fp32/fp16/fp8/int8-like values at fixed N, K and fixed frame sequence. Report temporal flicker, radiance bias, and edge leakage.
3. **KV-style sharing:** reuse static pages across 1, 2, 4, and 6 decode outputs. Report copy-on-write rate and stale-page error.
4. **Geometry invalidation:** deliberately introduce disocclusions, moving occluders, animated materials, and thin geometry. The cache passes only if hard gates prevent light leaks and ghost trails.

The investment decision is positive if block-paged or virtual-memory-backed radiance cache reduces memory waste and decode bandwidth without increasing visible stale-history artifacts. It is negative if geometry-aware invalidation forces so many page refreshes that the system behaves like full prefill every frame.

7.26 CUDA Data Layout and Binning

The first reconstruction implementation should be CUDA-first, but it should not depend on a CUDA acquisition backend existing yet. Recorded seed frames and CPU/Metal-produced seed frames are valid inputs as long as they satisfy the same device view. Once CUDA acquisition exists, it can alias or write those buffers directly without changing the reconstruction kernels.

Avoid a global nearest-neighbor search. Use a GPU-built hashed grid keyed by quantized position plus primitive ID:

$$h(x, id) = (\lfloor x_x/h_c \rfloor, \lfloor x_y/h_c \rfloor, \lfloor x_z/h_c \rfloor, id).$$

For each sample, search the cell and its 26 neighbors. Cap candidates per cell:

$$|B_h| \leq B_{max}.$$

If a cell overflows, reject low-confidence Gaussians first. If overflow persists, split the cell by primitive ID or material ID. The data layout should be structure-of-arrays so projection and update kernels can coalesce loads:

$$G = (\mu_x, \mu_y, \mu_z, \Sigma_0, \ldots, \Sigma_5, c_x, c_y, c_z, \nu, v, m, id_{prim}, id_{mat}, cell).$$

The CUDA staging buffers are:

- **sample buffers:** one entry per seed pixel/path event,
- **cell counts:** atomically incremented during binning,
- **cell offsets:** prefix-summed with CUB or an equivalent scan,
- **cell members:** compact Gaussian indices sorted or grouped by cell,
- **tile lists:** projected Gaussian ranges for TAA/SR/framegen,
- **debug counters:** gate rejections, atomics, overflows, clamps, prunes, and stale-history energy.

The prototype should measure atomics and memory traffic from the beginning. The research question is not only whether the equations work; it is whether the shared state maps naturally to CUDA.

7.27 CUDA Hash Grid Build

Build the hash grid with a standard count/scan/scatter pipeline:

1. `ComputeCellKeys`: one thread per Gaussian computes $h(\mu_j, id_j)$ and writes a compact cell key.
2. `CountCells`: atomically increments a cell count.
3. `ExclusiveScan`: converts counts to cell offsets.
4. `ScatterCellMembers`: writes Gaussian indices into `cell_members`.
5. `ClampOverflow`: marks excess low-confidence members for prune, split, or delayed update.

The per-cell occupancy distribution is a first-order performance signal:

$$\bar{B} = \frac{1}{N_c} \sum_h |B_h|, \qquad B_{p99} = \text{percentile}_{99}(|B_h|).$$

The method is healthy only if B_{p99} stays bounded under camera motion and moderate scene complexity. If a few cells dominate, assignment becomes divergent and tile projection develops long-tail latency.

7.28 CUDA Assignment and Update Strategy

Avoid updating Gaussian state directly from sample threads when possible. The simple atomic path is useful for M0, but the intended CUDA path is two-stage: sample-parallel assignment followed by Gaussian-parallel reduction.

Sample assignment writes:

$$A_i = (j_i^*, W_{ij_i^*}, x_i, c_i, L_i, n_i, u_i, \rho_i).$$

Then reduce by Gaussian ID:

$$S_j = \sum_{i:j_i^*=j} W_{ij}, \qquad X_j = \sum_{i:j_i^*=j} W_{ij} x_i, \qquad C_j = \sum_{i:j_i^*=j} W_{ij} c_i.$$

The update kernel becomes one thread block or warp group per active Gaussian:

$$\mu'_j = \frac{m_j \mu_j + X_j}{m_j + S_j + \epsilon}, \qquad c'_j = \frac{m_j c_j + C_j}{m_j + S_j + \epsilon}.$$

Radiance, velocity, and variance use the same reduced sums. This avoids many floating-point atomics to the same Gaussian and makes update cost depend on active Gaussians rather than raw samples. The first implementation can keep a compile-time switch:

- **atomic update path:** fastest to implement, useful for small scenes and debugging,
- **reduced update path:** intended performance path, using sorted assignments or bucketed per-cell reductions.

The experiment should report both. If the reduced path is required for quality because atomics create unstable history, that is acceptable. If it is required but too expensive, the shared-state CUDA design needs a different addressing scheme.

7.29 CUDA Spawn, Prune, Split, and Merge

Spawning should be queue-based:

$$q_i = \begin{cases} 1, & W_{ij^*} < \epsilon_W, \\ 0, & \text{otherwise.} \end{cases}$$

Prefix-sum q_i to allocate new Gaussian slots from a device free list:

$$j_{new} = F[\text{atomicAdd}(F_{head}, 1)].$$

Pruning writes dead Gaussian indices back to the free list:

$$dead_j = [\nu_j < \epsilon_\nu] \vee [t - \tau_j > T_{max}].$$

Split and merge should be separate compaction passes, not mixed into the sample update kernel. Split high-variance Gaussians by emitting two children along the largest tangent eigenvector:

$$\mu_{j\pm} = \mu_j \pm \kappa \sqrt{\lambda_{max}} e_{max}, \qquad \Sigma_{j\pm} = \eta_{split} \Sigma_j.$$

Merge is a low-frequency maintenance pass over local cells. The CUDA kill condition is simple: if split/merge must run every frame to avoid blur or explosive memory growth, the state representation is too fragile.

7.30 Tile List Construction and Reuse

Projection should use one tile list for all image-space consumers. For each Gaussian, compute its projected ellipse and conservative bounding box:

$$B_j = [p_j - r_j, p_j + r_j], \qquad r_j = k_\sigma \sqrt{\lambda_{max}(\Sigma_{screen,j})}.$$

The tile coverage is:

$$T_j = \{\, \tau : \tau \cap B_j \neq \emptyset \,\}.$$

Build tile lists with count/scan/scatter, mirroring the hash-grid build:

$$count_\tau = \sum_j [\tau \in T_j].$$

Then TAA, SR, feature projection, and frame generation consume the same `tile_members`. The consumers differ only in output resolution, confidence gates, and payload channels. A block-per-tile projection kernel can load a chunk of Gaussian indices into shared memory, evaluate per-pixel weights, and accumulate:

$$O_p = \frac{\sum_{j \in tile(p)} a_{pj} y_j}{\sum_{j \in tile(p)} a_{pj} + \epsilon}.$$

The crucial CUDA measurement is tile reuse:

$$R_{tile} = \frac{N_{consumers}\, N_{tile_members}}{N_{tile_members} + N_{extra_members}}.$$

If R_{tile} is close to one, the shared state is not buying much. If it is large and quality remains stable, vkGSplat has a concrete integration win.

7.31 Persistent State Versioning

Frame interpolation and temporal reconstruction need more than one state view. Maintain a ring buffer for compact motion-critical fields:

$$G_j^{hist}(r) = (\mu_j^r, \Sigma_j^r, c_j^r, \nu_j^r, \tau_j^r), \qquad r = t \bmod R.$$

The full radiance/path payload can stay single-buffered unless a path-space experiment needs old values. The CUDA memory rule is:

$$M_{history} = R\, M_{motion_core} + M_{payload}.$$

This is cheaper than duplicating the entire Gaussian state for TAA, SR, and frame generation. It also gives frame generation a clean source for $G_j(t_0)$ and $G_j(t_1)$ without keeping separate motion caches.

7.32 Default Prototype Parameters

The first demo should use explicit conservative defaults:

Parameter	Initial value	Purpose
ϵ_W	0.05	spawn threshold
ϵ_d	10^{-3}–10^{-2}	depth gate
γ_n	8	normal sharpness
η_{id}	0	hard primitive rejection
λ_{occ}	0.98	slow occlusion decay
λ_{dis}	0.2	fast stale-history decay
$\lambda_{frustum}$	0.995	preserve offscreen state briefly
m_{split}	32	split mass threshold
v_{split}	scene-dependent	split variance threshold
B_{max}	8–16	max Gaussians per hash cell

These values are not claims. They are a reproducibility anchor so vkGSplat can compare changes without silently moving the target.

7.33 Algorithm Summary

```
for each frame t:
    seed = acquire_seed_frame(t)  // CPU, Metal, CUDA RT/raster, or 3DGS
    upload_or_alias_seed_buffers(seed)

    launch ExtractSamples(seed, sample_buffers)
    launch UpdateVisibility(gaussians, seed.depth, seed.primitive_id)
    launch ComputeCellKeys(gaussians, cell_keys)
    launch CountCells(cell_keys, cell_counts)
    scan cell_counts -> cell_offsets
    launch ScatterCellMembers(cell_keys, cell_offsets, cell_members)

    launch AssignSamples(sample_buffers, cell_members, assignments, spawn_flags)
    scan spawn_flags -> spawn_offsets
    launch SpawnGaussians(spawn_offsets, free_list, gaussians)

    if use_reduced_update:
        sort_or_bucket assignments by gaussian_id
        launch ReduceAssignments(assignments, gaussian_deltas)
        launch ApplyGaussianDeltas(gaussian_deltas, gaussians)
    else:
        launch AtomicUpdateGaussians(assignments, gaussians)

    launch PruneAndCompactIfNeeded(gaussians, free_list)
    launch SplitOrMergeMaintenanceIfScheduled(gaussians)

    launch BuildTileLists(gaussians, camera_t, tile_members)
    launch ProjectTAA(tile_members, gaussians, taa_frame)
    launch ProjectSR(tile_members, gaussians, sr_frame)
    launch ProjectFeatures(tile_members, gaussians, features)

    if path_tracing:
        launch QueryRadiance(gaussians, path_queries)
        launch UpdatePathSubslots(path_events, gaussians)

    if interpolation:
        launch SynthesizeGenerated(tile_members, gaussians, alpha, framegen_out)
```

7.34 CUDA Kernel Decomposition

The CUDA prototype should be decomposed into explicit kernels so each mathematical claim maps to a launch, a memory access pattern, and a measurable cost:

CUDA kernel	Input	Output	Main risk
ExtractSamples	seed frame	S_t	bandwidth
BinGaussians	G_j	cell lists	atomics
AssignSamples	S_t, cells	assignments	divergence
UpdateMoments	assignments	updated G_j	write conflicts
UpdateVisibility	depth, IDs	ℓ_j^t, ν_j	false stale kills
BuildTileLists	projected G_j	tile ranges	overdraw
ProjectTAA	tile lists	history/confidence	depth leaks
ProjectSR	tile lists, scale	high-res resolve	bandwidth
QueryRadiance	path events	$\hat{L}^k$	incoherent reads
UpdatePathSubslots	assignments	A_j^{path}	noisy atomics
SynthesizeFrame	G_j^α	generated frame	holes
ProjectFeatures	tile lists	feature planes	channel cost

The CUDA kernels should be deterministic for fixed input where possible, but perfect CPU-like determinism is not the objective. The objective is to expose where the unified Gaussian state pays for itself on the GPU: fewer duplicated history buffers, fewer independent passes, reusable tile lists, and shared confidence/visibility logic.

7.35 CUDA Scheduling

The preferred frame schedule is:

1. seed frame and auxiliary buffers are produced by any acquisition backend that satisfies the seed ABI,
2. `ExtractSamples` writes a compact sample stream,
3. `UpdateVisibility` marks visible/occluded/stale Gaussians,
4. `BinGaussians` and `AssignSamples` update the persistent state,
5. `BuildTileLists` constructs one projection structure,
6. TAA, SR, feature projection, and frame generation consume the same tile lists,
7. radiance queries update path subslots asynchronously when path tracing samples are available.

This schedule is the real integration claim. If TAA, SR, radiance, and frame generation require separate binning/projection structures, then 3DGS remains a collection of useful tricks rather than vkGSplat's central CUDA abstraction. If they require a specific acquisition renderer, the software stack has failed at an even earlier boundary.

7.36 Task-to-Experiment Map

Task	Equation family	First experiment	Failure signal
TAA	H_t, β_p	Exp. 1	shimmer or ghosting
2x/4x SR	O_t^{SR}	Exp. 1	blur or scale-specific state
Denoising	F_L, V_L	Exp. 2	variance unchanged
Radiance cache	$\hat{L}^k$	Exp. 2	light leaking
Path-space CV	$\hat{I}_{CV}$	Exp. 2	bias exceeds variance gain
Interpolation	μ^α, ν^α	Exp. 3	silhouette error
Frame generation	H_α, E_{FG}	Exp. 3	confident holes
Neural resolve	Φ_t, N_θ	Exp. 4	features ignored

This table is the falsifiability guardrail. If an equation does not land in an experiment, it should not be in the first implementation.

8 Feature Integration

8.1 Monte Carlo Denoising and Radiance Caching

Use path samples to update radiance-bearing Gaussian levels:

- level 0: directly visible surface evidence,
- level 1: first-bounce diffuse or rough indirect radiance,
- level 2: deeper low-frequency residual,
- residual level: optional control-variate correction.

The cache is queried during low-spp rendering and also projected as a denoising feature. This tests whether radiance history and image history can be one state.

8.2 Temporal AA

The Gaussian state acts as a world-space history buffer. Instead of sampling previous-frame pixels, the current frame projects persistent Gaussians into the current camera and accepts them only when depth, primitive ID, normal, velocity, and confidence agree.

8.3 Temporal Super-Resolution

Jittered low-resolution samples update Gaussian footprints. The output image is resolved at target resolution by splatting projected footprints. The key question is whether the state preserves subpixel evidence better than a well-clamped image-space TAAU baseline.

8.4 Motion-Accurate Frame Interpolation

For interpolation between t_0 and t_1, evaluate each moving Gaussian at t_α:

$$\mu_j(t_\alpha) = (1-\alpha)\mu_j(t_0) + \alpha\mu_j(t_1)$$

for the first prototype. Later versions use object transforms, splines, Kalman-style prediction/correction, or flow-guided residuals. Motion is trusted only when projected forward/backward residuals are small:

$$e_j = \|\Pi_{t_1}(\mu_j(t_1)) - (\Pi_{t_0}(\mu_j(t_0)) + u_{0\to 1})\| \,.$$

This is stronger than plausible frame generation: the midpoint should align with a true rendered middle frame.

8.5 Hybrid Learned Resolver

A small neural resolver is optional later work. It can consume both normal engine buffers and Gaussian features:

$$\text{low-res frame} + \text{engine buffers} + \text{Gaussian features} \to \text{reconstructed frame}.$$

This does not replace the explicit state, and it is not required for the core framework. It tests whether the explicit state reduces the amount of learning needed.

9 What Can Approach DLSS 4.5 or DLSS 5

Likely approachable in vkGSplat:

- improved temporal stability over naive reprojection,
- 2x temporal SR for controlled scenes,
- motion-accurate midpoint interpolation for rigid camera/object motion,
- radiance-cache-assisted diffuse indirect reconstruction,
- explicit world-space conditioning for a later optional neural lighting/material residual,
- a hybrid neural resolver that uses Gaussian features, if the no-training core already shows signal.

Not likely approachable soon:

- generic per-game DLSS 4.5 product quality,
- DLSS 5 photoreal material/lighting synthesis without a large trained model, art controls, and production validation,
- robust multi-frame generation for particles, UI, transparency, and specular shading without an image-space learned residual,
- DLSS-class latency integration without Reflex-like scheduling,
- parity with proprietary NVIDIA training and validation infrastructure.

The useful vkGSplat question is therefore not "can 3DGS replace DLSS?" It is: can an explicit Gaussian state provide a better conditioning signal than color, depth, motion vectors, and exposure alone? If yes, vkGSplat becomes a plausible open research route toward a DLSS-class stack: analytic Gaussian history first, then, only if useful, a small neural residual that receives projected Gaussian features.

10 Metrics and Pass/Fail Rules

The experiments should not rely on visual inspection alone. The shared-state hypothesis needs a scorecard that measures quality, stability, integration, and cost.

10.1 Image and Temporal Error

Reference error:

$$E_{ref} = \frac{1}{|\Omega|} \sum_{p \in \Omega} \|O_t(p) - R_t(p)\|_2^2.$$

Temporal instability after reprojection:

$$E_{temp} = \frac{1}{|\Omega_v|} \sum_{p \in \Omega_v} \|O_t(p) - O_{t-1}(p + u_{t \to t-1})\|_1.$$

Disoccluded pixels are excluded from Ω_v and scored separately.

10.2 Ghosting and Disocclusion

Given a ground-truth or depth/primitive-derived disocclusion mask $D_t(p)$, ghosting error is:

$$E_{ghost} = \frac{1}{|D_t|} \sum_{p: D_t(p)=1} \|O_t(p) - C_t(p)\|_1.$$

This penalizes stale history in newly revealed regions. A Gaussian method that improves PSNR but raises E_{ghost} is not acceptable for the first demo.

10.3 Interpolation Accuracy

For midpoint interpolation, compare against a true rendered middle frame:

$$E_{mid} = \frac{1}{|\Omega|} \sum_{p} \|O^{gen}_{t+\frac{1}{2}}(p) - R_{t+\frac{1}{2}}(p)\|_2^2.$$

Silhouette error uses depth or primitive masks:

$$E_{sil} = 1 - \frac{|S_{gen} \cap S_{ref}|}{|S_{gen} \cup S_{ref}|}.$$

The midpoint is motion-accurate only if both E_{mid} and E_{sil} improve over image blending and screen-space reprojection.

10.4 Integration Score

Define a normalized per-feature improvement over the image-space baseline:

$$I_f = \frac{E_f^{baseline} - E_f^{method}}{E_f^{baseline} + \epsilon}.$$

For features $f \in \{\text{TAA}, \text{SR}, \text{Denoise}, \text{Interp}\}$:

$$I_{shared} = \frac{1}{4}\sum_f I_f - \lambda_C \frac{C_{method}}{C_{baseline}} - \lambda_M \frac{M_{method}}{M_{baseline}}.$$

The shared Gaussian state passes the integration test if:

- at least three of four I_f are positive,
- no ghosting metric regresses by more than 10%,
- shared-state cost is below separate-Gaussian cost,
- shared-state memory is within 1.5x of the image-space baseline for the first CUDA demo, or clearly on a path to that bound.

11 Decisive Experiments

11.1 Experiment 0: Baseline Reconstruction Stack

Purpose. Establish honest non-Gaussian baselines.

Inputs. Low-spp or low-resolution seed frames with color, depth, NDC depth, primitive ID, camera matrices, jitter, and optional motion vectors.

Baselines.

- current vkGSplat reprojection,
- TAA with neighborhood clamp,
- TAAU 2x with jitter,
- SVGF-style denoising,
- optical-flow interpolation if available.

Metrics. PSNR, SSIM, temporal difference, edge shimmer, ghosting after disocclusion, silhouette error, runtime, and memory.

Expected signal. This defines the bar. Gaussian results are meaningful only if these baselines are not straw men.

11.2 Experiment 1: Single Gaussian State, Multiple Outputs

Purpose. Demonstrate integration directly.

Scene. Static Cornell-style scene with thin geometry, textured surfaces, camera jitter, and slow camera pan.

Method. Build one Gaussian state from visible samples. Use that same state to produce:

- TAA history,
- 2x super-resolution resolve,
- variance/confidence feature frame,
- disocclusion mask.

Baselines.

- separate image-space TAA plus TAAU,
- Gaussian TAA only,
- Gaussian SR only,
- shared Gaussian state for both.

Metrics.

- TAA shimmer reduction,
- SR reference error,
- ghosting score,
- number of duplicated state buffers,
- update/query cost.

Expected success. The shared Gaussian state performs within 5–10% of specialized per-feature Gaussian variants while beating image-space history on shimmer or disocclusion.

Failure case. If each feature requires incompatible covariance, confidence, or filtering rules, 3DGS is not a clean integration layer.

11.3 Experiment 2: Gaussian Radiance Cache Plus TAA

Purpose. Test whether lighting history and image history can share the same Gaussian state.

Scene. Diffuse Cornell box plus one rough glossy variant.

Method. Add path sample logging. Store level-1 indirect radiance in the same Gaussian entries or in path-depth subslots attached to the same entry. Project color history for TAA and query indirect radiance for low-spp shading.

Baselines.

- path tracing plus SVGF,
- Gaussian TAA without radiance cache,
- separate radiance cache plus separate TAA history,
- unified Gaussian state with color and radiance subslots.

Metrics. Indirect-light MSE, variance at fixed spp, temporal stability, light leaking, stale lighting after light changes, runtime, and memory.

Expected success. Unified state reduces variance and temporal shimmer without requiring a separate cache structure.

Failure case. Radiance and color history need different support sizes or gating so strongly that they should not be one state.

11.4 Experiment 3: Motion-Accurate Midpoint From Same State

Purpose. Test whether the temporal/SR state can also drive frame interpolation.

Scene. Render t_0, $t_{0.5}$, and t_1 for:

- camera-only motion,
- one rigid moving object,
- one simple deforming object.

Method. Hide $t_{0.5}$. Use Gaussian motion state to synthesize the midpoint. Compare against the true middle frame.

Baselines.

- frame hold,
- linear image blend,
- screen-space reprojection,
- optical-flow interpolation if available.

Metrics. Midpoint PSNR/SSIM, silhouette distance, depth-edge alignment, occlusion error, flow residual, and temporal flicker across t_0, generated $t_{0.5}$, t_1.

Expected success. Rigid and camera-motion midpoint silhouettes land closer to ground truth than screen-space methods.

Failure case. Disoccluded background and non-rigid motion require a separate residual system, weakening the integration thesis.

11.5 Experiment 4: Unified State vs Separate State Ablation

Purpose. Directly answer whether 3DGS integrates features well.

Variants.

- image-space-only stack,
- separate Gaussian state per feature,
- one shared Gaussian state with task-specific subslots,
- one shared Gaussian state plus small neural resolver.

Scorecard.

Variant	TAA	SR	Denoise	Interp	Cost
Image-space only	?	?	?	?	?
Separate Gaussians	?	?	?	?	?
Shared Gaussian state	?	?	?	?	?
Shared state + neural resolver	?	?	?	?	?

Decision rule. The shared Gaussian state is a good integration layer if it wins on at least three of four quality axes while keeping memory/runtime within roughly 1.5x of the image-space baseline or below the separate-Gaussian variant.

Ablation protocol. Keep sample inputs, camera jitter, references, and post-filters identical. Only change the reconstruction state:

1. **Image-space only:** color/depth/velocity history, no Gaussians.
2. **Separate Gaussians:** one Gaussian cache for TAA/SR, one for radiance, one for interpolation.
3. **Shared Gaussians:** one state with task-specific subslots.
4. **Shared plus neural:** same shared state, plus equal-size neural residual model.

The separate-Gaussian variant is important. If it beats the shared state by a large margin, then 3DGS may still be useful, but not as the unifying vkGSplat abstraction.

11.6 Experiment 5: Downstream Synthetic Data Utility

Purpose. Test whether reconstruction quality matters for vkGSplat's synthetic-data goal.

Method. Generate a small robotics/perception dataset with:

- noisy low-spp images,
- image-space denoised/TAAU images,
- unified Gaussian reconstructed images,
- high-spp/high-res references.

Metrics. Downstream detection/segmentation/pose accuracy, not only image PSNR.

Expected success. The Gaussian reconstruction improves downstream accuracy or data efficiency relative to noisy and image-space baselines.

12 CUDA Validation and Microbenchmarks

The first implementation should be a CUDA prototype with validation and microbenchmarks from day one. The tests are wrappers around GPU kernels, not a CPU research path. A scalar CPU oracle is useful only for tiny deterministic inputs and expected-value checks.

- **cuda_extract_samples:** device kernel unprojects depth, primitive ID, color, and motion into compact sample buffers.
- **cuda_bin_gaussians:** device hash-grid binning reports cell occupancy, overflow rate, atomic pressure, and memory bandwidth.
- **cuda_assign_and_update:** incompatible primitive, depth, normal, and motion samples do not update the same Gaussian; benchmark branch divergence and update atomics.
- **cuda_visibility_decay:** occluded Gaussians decay slowly while disoccluded stale Gaussians decay quickly on device.

- **cuda_project_taa_static_pan:** shared-state TAA reduces temporal error on a jittered pan while reporting tile-list reuse.
- **cuda_project_sr_scale2_4:** the same state resolves 2x and 4x outputs using only changed projection footprints.
- **cuda_radiance_diffuse:** level-1 diffuse radiance cache reduces variance against a high-spp Cornell reference without crossing primitive boundaries.
- **cuda_path_subslot_moments:** path-depth/lobe moments update on device and biased mode is separated from residual/control-variate mode.
- **cuda_midpoint_rigid:** midpoint interpolation lands a moving rigid object's silhouette closer to the rendered middle frame than linear image blending.
- **cuda_framegen_holes:** disoccluded generated pixels are marked as holes and do not receive high-confidence Gaussian color.
- **cuda_shared_vs_separate:** shared-state memory and launch count are lower than separate per-feature Gaussian caches while quality stays within the scorecard threshold.

These benchmarks should use tiny synthetic scenes first, then Wicked/Cornell captures once the CUDA contracts are stable. Report timing with CUDA events and memory counters, not only image quality.

13 Implementation Milestones

13.1 M0: CUDA State Without New Rendering

Start with existing vkGSplat frames and fixtures, but implement the Gaussian state path as CUDA kernels. Add only device sample extraction, binning, assignment/update, visibility, and projection. This avoids confusing reconstruction quality with renderer changes while keeping the research pointed at the real target: GPU integration.

Exit criteria:

- CUDA sample extraction matches the small CPU oracle on tiny inputs,
- hash-grid binning has bounded overflow and measurable atomic cost,
- visibility decay behaves differently for occlusion and disocclusion,
- all kernels expose timing, bandwidth, and debug counters.

13.2 M1: Shared-State TAA

Implement only C_j and A_j^{taa}. Use static camera-pan clips with jitter.

Exit criteria:

- temporal error improves over current reprojection,
- ghosting does not regress by more than 10%,
- stale history disappears in disoccluded regions within two frames,
- TAA projection reuses the same CUDA tile/binning data intended for SR.

13.3 M2: Same State, 2x/4x SR

Add A_j^{sr} but keep the same shared core C_j. The only scale-specific change should be the projection footprint.

Exit criteria:

- 2x SR beats bilinear and screen-space TAAU on thin geometry,
- 4x SR does not require a second Gaussian cache,
- blur is bounded by an edge-width metric against high-res reference,
- CUDA projection cost scales with output pixels and tile occupancy, not by rebuilding the Gaussian state.

13.4 M3: Radiance Subslot

Add A_j^{rad} for level-1 diffuse/rough indirect radiance. Keep direct light sampled normally.

Exit criteria:

- variance drops at fixed spp,
- light leaking across primitive/depth boundaries stays below baseline denoiser artifacts,
- TAA/SR quality does not degrade when radiance subslots are enabled,
- radiance subslot updates do not dominate frame time through atomics.

13.5 M4: Motion Subslot

Add A_j^{mot} and synthesize true midpoint frames for rigid object and camera-only motion.

Exit criteria:

- silhouette IoU beats image blending and screen reprojection,
- flow residual predicts failures,
- disoccluded background is masked rather than hallucinated as confident,
- generated frames reuse Gaussian motion/tile data rather than launching an unrelated image-space interpolation stack.

13.6 M5: Shared State Plus Neural Residual

Only after M1–M4 show signal, train a small equal-size resolver with and without Gaussian feature planes.

Exit criteria:

- Gaussian features improve error at fixed model size,
- model does not simply learn to ignore Φ_t,
- inference remains a small residual pass, not the whole system.

14 CUDA Implementation Sketch

Start with CUDA-facing contracts. Host structs define buffer layout and launch configuration; the persistent Gaussian state lives in device structure-of-arrays buffers:

```
struct GaussianSampleDeviceView {
    float3* position;
    float3* normal;
    float3* color;
    float3* radiance;
    float3* velocity;
    float2* motion_px;
    float* depth;
    float* variance;
    float* roughness;
    uint32_t* primitive_id;
    uint32_t* material_id;
    uint32_t* path_depth;
    uint32_t count;
};

struct PathLobeMoments {
    float3 mean;
    float second_moment;
    float sample_weight;
    float robust_residual;
};

struct GaussianStateDeviceView {
    float3* mean;
    float4* covariance_0_3;
    float2* covariance_4_5;
    float4* rotation;
    float3* color;
    float3* radiance_levels[3];
    PathLobeMoments* path_lobes;
    float3* velocity;
    float* variance;
    float* confidence;
    float* visibility_age;
    float* flow_residual_px;
    uint32_t* primitive_id;
    uint32_t* material_id;
    uint32_t* motion_group_id;
    uint32_t count;
};
```

CUDA launch API:

```
class CudaGaussianReconstructionCache {
public:
    void extract_samples_cuda(const SeedFrameDeviceView& seed);
    void update_visibility_cuda(const DepthIdDeviceView& current);
    void bin_gaussians_cuda();
    void assign_and_update_cuda();
    void build_tile_lists_cuda(const CameraDeviceConstants& camera);
    void project_taa_cuda(ReconstructionFrameDeviceView out) const;
    void project_super_resolution_cuda(ReconstructionFrameDeviceView out) const;
    void query_indirect_cuda(ShadingQueryDeviceView queries) const;
    void update_path_subslots_cuda(PathEventDeviceView events);
    void synthesize_generated_cuda(float alpha, FrameDeviceView out) const;
    void project_features_cuda(FeatureFrameDeviceView out) const;
};
```

The first public C++ API can wrap these launches, but the design should be judged by CUDA memory layout, launch count, and tile-list reuse.

15 Numerical Invariants and Debug Counters

The first CUDA implementation should treat every equation above as an invariant that can fail loudly. This makes the research question falsifiable inside the GPU path, not after translating a CPU prototype.

15.1 Hard Invariants

1. **Confidence is bounded:**

$$0 \leq \nu_j \leq 1.$$

   Any update that produces a value outside this range is a bug, not a tuning problem.

2. **Covariance stays positive definite:**

$$\lambda_{min} I \preceq \Sigma_j \preceq \lambda_{max} I.$$

   Clamp eigenvalues in the CUDA update kernel after every update. A clamped covariance should increment a device debug counter; frequent clamps mean assignment or split rules are unstable.

3. **Primitive identity is a hard early gate:**

$$id_i^{prim} \neq id_j^{prim} \quad \Rightarrow \quad W_{ij} = 0$$

   when both IDs are valid. Later experiments can soften this for alpha-tested or deforming surfaces, but the first demo should prove that 3DGS history does not cross triangle boundaries.

4. **Disoccluded history is never high confidence:**

$$D_p = 1 \Rightarrow \sum_{j \to p} a_{jp} \nu_j \leq \epsilon_D.$$

   This is the main anti-ghosting invariant. If a newly revealed pixel receives strong old Gaussian history, the shared-state thesis fails for TAA.

5. **Radiance estimates stay non-negative and bounded:**

$$0 \leq \hat{L}_j^k \leq L_{max,k}.$$

   The bound is not a physically pure estimator; it is a prototype guard against rare path samples poisoning TAA/SR history.

6. **Stable assignment:** if two Gaussians have equal compatibility, the lower stable index wins. This keeps CUDA validation reproducible enough to compare runs while accepting normal floating-point ordering differences.

15.2 Per-Frame Counters

Every run should report:

- samples extracted, assigned, rejected, and spawned,
- rejections by gate: primitive, depth, normal, motion, residual, confidence,
- covariance clamps, splits, merges, and prunes,
- visible, occluded, disoccluded, and out-of-frustum Gaussian counts,
- stale-history energy on disoccluded pixels,
- memory bytes for shared state versus separate-feature states,
- per-feature quality deltas: TAA, SR, radiance, interpolation.

The key diagnostic is stale-history energy:

$$E_{stale} = \frac{1}{|D_t|} \sum_{p:D_t(p)=1} \left\| \sum_{j \to p} a_{jp} \nu_j c_j \right\|_1 .$$

If E_{stale} stays high after tuning gates, world-space Gaussians are not a safe TAA history representation for the current scene class.

16 CUDA Performance Targets

Quality wins are not enough. A shared Gaussian state is valuable only if the CUDA execution graph is simpler or cheaper than separate feature stacks.

16.1 Primary GPU Metrics

Each experiment should report:

- per-kernel time from CUDA events,
- end-to-end reconstruction frame time,
- global memory read/write bytes,
- atomics per sample and atomics per Gaussian,

- hash-cell overflow rate,
- tile-list build cost and tile-list reuse count,
- average and worst-case Gaussians per tile,
- occupancy, register pressure, and shared-memory use for update and projection kernels,
- memory footprint for image-space baselines, separate Gaussian states, and the shared Gaussian state.

16.2 GPU Integration Score

The CUDA-specific score should separate visual quality from implementation cost:

$$S_{cuda} = S_{quality} - \lambda_t \frac{T_{shared}}{T_{image}} - \lambda_m \frac{M_{shared}}{M_{image}} + \lambda_r R_{reuse}.$$

R_{reuse} measures how often the same Gaussian bins, tile lists, confidence values, and visibility decisions feed multiple outputs:

$$R_{reuse} = \frac{N_{shared_consumers}}{N_{extra_builds} + 1}.$$

The shared-state thesis is weak if $S_{quality} > 0$ but $S_{cuda} < 0$. That case means the idea may be mathematically interesting but not a good vkGSplat CUDA architecture.

16.3 CUDA Kill Conditions

Stop or redesign the architecture if:

- assignment requires unbounded neighbor search,
- atomics dominate update time before scenes become interesting,
- TAA, SR, radiance, and frame generation each rebuild their own bins,
- tile occupancy variance creates persistent long-tail frame times,
- shared-state memory exceeds separate Gaussian states,
- quality depends on a CPU-only ordering or exact determinism assumption.

17 Parameter Sweeps and Thesis Kill Tests

The first implementation should not hand-tune one pleasant clip. It should run small sweeps over the parameters that decide whether the unified state is actually robust.

Parameter	Sweep	Primary metric	Kill condition
θ_{assign}	0.05–0.60	E_{ghost}, E_{temp}	no stable interval
σ_d	0.002–0.08	disocclusion error	depth leak unavoidable
σ_n	0.05–0.50	edge shimmer	normal gate unused
λ_{min}	10^{-6}–10^{-3}	blur, clamps	covariance collapses
ρ_{SR}	0.35–1.00	edge width	2x requires new state
γ_{dis}	0.05–0.50	stale energy	stale energy persists
K_{cell}	8–64	memory, quality	quality needs unbounded K
$\eta_{\dot{\mu}}$	0.05–0.80	E_{mid}, E_{sil}	residual cannot predict fail

For each sweep point, compute a compact integration score:

$$S = w_T \Delta E_{temp} + w_R \Delta E_{ref}^{SR} + w_L \Delta V_L + w_M \Delta E_{mid} - w_G \max(0, \Delta E_{ghost}) - w_C \Delta C_{cost}.$$

Here positive Δ means improvement over the image-space baseline, except for ghosting and cost where increases are penalties. The unified state passes the research bar only if there is a non-tiny parameter interval where $S > 0$ and E_{ghost} remains within the 10% guardrail. A single winning parameter point is not enough.

17.1 Ablation Protocol

The decisive ablation should hold sample inputs, jitter, and camera path fixed:

1. **Image-space only:** current reprojection plus SVGF-style denoise.
2. **Separate Gaussian states:** one Gaussian cache for TAA, one for SR, one for radiance, one for motion.
3. **Shared core:** one C_j with task subslots.
4. **Shared core plus neural residual:** same as above, with projected Gaussian feature planes.

The integration hypothesis is supported only if variant 3 beats variant 2 on cost or consistency while staying close in quality. Variant 4 is useful only if the learned residual uses Gaussian features; test this by zeroing Φ_t at validation time and measuring the error jump.

18 Minimal Demonstration in the Current vkGSplat Workspace

The lowest-risk implementation path fits the current repository shape:

- add `include/vkgsplat/cuda/gaussian_reconstruction.h`,
- add `src/cuda/gaussian_reconstruction.cu`,
- add `tests/test_cuda_gaussian_reconstruction.cu`,
- register the CUDA benchmark/test only under `VKGSPLAT_ENABLE_CUDA`.

The first CUDA code should reuse existing frame contracts instead of inventing new scene loading:

- `RayTracingSeedFrame` supplies color, depth, NDC depth, primitive ID, and camera matrices,
- `compute_camera_motion_map` supplies the baseline motion map,
- `reproject_history` supplies the image-space TAA baseline,
- `denoise_svgf_baseline` supplies the Monte Carlo denoise baseline.

The first demo sequence should be tiny:

1. render four low-resolution jittered seed frames from the existing triangle/Cornell-style ray-tracing seed path,
2. upload seed frames and auxiliary buffers to device memory,
3. build the Gaussian cache from frames 0–2 using CUDA update kernels,
4. project frame 3 at native resolution for TAA comparison,
5. project frame 3 at 2x resolution for SR comparison,
6. synthesize the midpoint between frames 1 and 2 on device, comparing against a held-out rendered middle frame,
7. optionally query level-1 radiance from the same Gaussian state for a low-spp diffuse indirect test,
8. report CUDA launch time, memory traffic, atomics, tile occupancy, and quality metrics for every run.

This demo proves integration only if the same Gaussian indices and shared core fields are used across all projections. A convenient implementation check is:

$$\frac{|J_{taa} \cap J_{sr} \cap J_{mot}|}{|J_{taa} \cup J_{sr} \cup J_{mot}|} \geq 0.6$$

where J_{task} is the set of Gaussian IDs that materially contribute to a task output. If overlap is low, the method is drifting toward separate hidden systems.

19 DLSS-5-Style Control Plane

If the analytic state works, the later neural resolver should not receive only RGB history. It should receive a control plane that makes generated detail accountable to the explicit scene state:

$$X_t = (C_t, Z_t, U_t, N_t, A_t, \Phi_t^{gauss}, \nu_t^{gauss}, V_t^{gauss}, D_t).$$

Here C_t is low-resolution color, Z_t depth, U_t motion, N_t normal, A_t material/albedo metadata, Φ_t^{gauss} projected Gaussian features, ν_t^{gauss} confidence, V_t^{gauss} variance, and D_t disocclusion. The resolver output should be decomposed:

$$O_t = O_t^{analytic} + R_\theta(X_t),$$

where $O_t^{analytic}$ is the deterministic Gaussian reconstruction and R_θ is a bounded residual. This keeps the DLSS-5-style part honest: the network can add high-frequency lighting/material corrections, but the analytic state remains responsible for geometry, motion, visibility, and uncertainty.

20 Risks

- One state may be too overloaded; TAA, radiance, and interpolation may need incompatible covariance scales.
- Ghosting may worsen if stale Gaussians persist after disocclusion.
- Radiance caching can leak light across visibility boundaries.
- Super-resolution can become stable but blurry.
- Frame interpolation needs residual handling for particles, UI, transparency, reflections, and animated shading.
- A DLSS-class commercial product still needs large training, validation, and latency engineering beyond the explicit Gaussian state, but the core vkGSplat framework should not depend on that.

21 Current Recommendation

Proceed with the unified 3DGS integration thesis. The first target should not be a DLSS clone. The first target should be a falsifiable demo:

> One Gaussian state, built from low-resolution jittered seed frames, drives both TAA and 2x super-resolution on a thin-geometry camera-pan clip, then reuses the same state for a rigid-motion midpoint interpolation test.

If that works, add radiance-cache subslots. Only after the no-training path has a measurable signal should vkGSplat add a small learned resolver. If it does not, 3DGS may still be useful for individual features, but not as the integrating idea behind vkGSplat.

A Appendix: Starting from Zero Graphics Hardware

If a vkGSplat-oriented accelerator starts from zero graphics-specific hardware, it should not attempt to recreate a full conventional GPU pipeline first. The right target is a neural-rendering/GS accelerator with a minimal graphics-like prefill front end. Graphics hardware should be treated as an evidence generator, not as the center of the product.

The minimum useful prefill hardware is:

- tiled triangle or mesh rasterization for visibility, depth, primitive identity, and barycentrics,
- depth test and preferably hierarchical/tile Z,
- barycentric attribute interpolation for position, normal, material, and motion features,
- basic texture sampling only if material evidence cannot be supplied another way,
- simple compositing/blending for seed buffers,
- optional BVH traversal only if ray evidence is central to the product.

The blocks to avoid in a first architecture are the broad legacy pieces that exist mostly to support full graphics API completeness:

- a complex fixed-function shader pipeline,
- tessellation, geometry-shader, and legacy compatibility paths,
- a complete texture/filtering ecosystem beyond the needed evidence channels,
- a large ROP/blend feature matrix,
- heavyweight generalized graphics scheduling,
- large RT traversal blocks unless radiance/path evidence is a primary workload.

The evidence target is not a finished shaded frame. The target is a tensor bundle:

$$A_t = (L, D, ID, X, N, V, M, \nu, \sigma^2),$$

where L is color/radiance evidence, D is depth, ID is primitive/material identity, X is world position, N is normal, V is motion, M is material or mask data, ν is confidence, and σ^2 is variance. Once this bundle is trustworthy, the differentiating work moves into Gaussian update, radiance-cache paging, tensorized decode, and optional neural resolving.

For a first vkGSplat-class architecture, prioritize hardware roughly as:

Priority	Hardware block	Reason
1	tensor/matrix engines	Gaussian feature projection, attention-like resolve, and neural decode
2	SRAM/cache bandwidth and paging	radiance/KV-like cache residency and streaming reuse
3	sparse gather/scatter, reductions, atomics	online Gaussian update, compaction, assignment, and binning
4	tile scheduler and block/page tables	local decode, work clustering, and cache management
5	minimal raster/depth/interpolation	trustworthy prefill evidence for visibility, IDs, depth, and motion
6	texture sampling	only where material evidence cannot be represented cheaply as attributes
7	BVH/ray traversal	only if path/radiance evidence is central to the product target

The architectural pipeline should be:

$$\text{minimal raster/ray prefill} \rightarrow \text{seed evidence tensors} \rightarrow \text{Gaussian update}$$
$$\rightarrow \text{paged radiance cache} \rightarrow \text{tensor decode for TAA/SR/framegen.}$$

This is different from dropping graphics entirely. The accelerator still needs visibility, identity, depth, and material evidence. But it does not need to spend early area on a general-purpose graphics pipeline if the product thesis is persistent Gaussian state and DLSS-class decode. The design principle is:

> Do not reinvent graphics hardware wholesale. Reinvent the boundary: a thin prefill evidence generator feeding a tensorized Gaussian/radiance-cache decode engine.

A useful pass/fail test for hardware planning is to implement two simulators:

1. **Minimal evidence prefill:** raster/depth/ID/interpolation only, plus tensor decode.
2. **Richer traditional prefill:** add texture/filtering/ray features progressively.

If richer prefill improves reconstruction quality much more than it costs, add those graphics blocks. If the minimal prefill already gives stable Gaussian state and good decode quality, invest the saved area in tensor/cache/paging instead. The burden of proof should be on adding legacy graphics area, not on removing the core tensor/cache path.

B Appendix: Architectural Path Dependence

The deeper vkGSplat thesis is not only that 3DGS can run on tensor cores. It is that computer architecture is shaped by dominant paradigms, and then the resulting programmable substrate becomes available to later paradigms. A successful workload creates hardware pressure; hardware vendors respond with specialized but still programmable mechanisms; a different representation later discovers that those mechanisms are close to the right machine for its own needs.

A compact form of the pattern is:

$$\text{dominant paradigm} \rightarrow \text{hardware specialization}$$
$$\rightarrow \text{programmable affordance} \rightarrow \text{new paradigm reuse.}$$

Historically, raster graphics did this first. The need to draw shaded triangles at high throughput justified programmable GPU shaders, high memory bandwidth, texture units, and massive data parallelism. Once exposed as a programmable substrate, that graphics-shaped machine became useful for scientific computing, deep learning, differentiable rendering, NeRFs, and other non-raster workloads. The hardware was motivated by graphics, but it did not remain confined to classical graphics.

Transformer/LLM serving is now applying the same pressure in a different region of the design space. It justifies tensor engines, FP8/FP4 arithmetic, block-scaled quantization, KV-cache paging, high-bandwidth memory, large SRAM, continuous batching, and decode-oriented scheduling [31, 32, 34, 36, 37]. These features are not graphics-specific, but they are surprisingly well matched to Gaussian-state reconstruction: tile-local feature projection, paged radiance caches, motion-aligned history, low-precision feature packets, sparse gather/reduce, and optional transformer decoding.

Dominant paradigm	Architecture it encourages	Later reuse
Raster graphics	Programmable shaders, high bandwidth, texture access, SIMT execution	GPGPU, deep learning, differentiable rendering, neural fields
Deep learning and LLMs	Tensor cores, FP8/FP4, HBM, SRAM, cache paging, batched decode	3DGS feature projection, radiance cache, TAA/SR/framegen decode
Stateful 3DGS reconstruction	Gaussian/tile pages, sparse updates, visibility-aware caches, narrow evidence prefill	Future digital twins, robotics SDG, neural-rendering accelerators

For vkGSplat, the inversion is important. Graphics hardware helped create the GPU. AI then reshaped the GPU around tensor and cache workloads. Now a new graphics representation,

3D Gaussian Splatting, may benefit more from the AI-shaped architecture than from the older fixed-function graphics pipeline. 3DGS is therefore not competing with LLM hardware. It is discovering that LLM hardware has already built much of the right substrate for stateful graphics reconstruction.

This view also explains why vkGSplat should not begin by cloning a full legacy GPU. The prefill side still needs graphics-like evidence: visibility, depth, primitive identity, barycentrics, material hints, normals, and motion. But the decode side is where the architectural surplus lies. Gaussian state update, radiance-cache lookup, projected feature tokens, temporal reconstruction, super-resolution, and frame generation all map to the tensor/cache/paging machinery that LLM serving made economically inevitable.

The appendix-level claim is therefore:

> vkGSplat is an example of architectural path dependence. Transformer/LLM serving justifies hardware investment in tensor engines, low-precision arithmetic, cache paging, and high-throughput decode scheduling. Those investments then become programmable affordances for a different paradigm: Gaussian-state graphics reconstruction.

The falsifiable version is the same as the hardware ablation in the main text. If a minimal evidence prefill plus strong tensor/cache decode beats a richer graphics prefill under equal area, bandwidth, or energy budgets, then vkGSplat is benefiting from LLM-shaped hardware. If richer prefill dominates, then the old graphics paradigm still owns the bottleneck. The interesting outcome is not ideological; it is the measured boundary where one dominant paradigm's hardware surplus becomes another paradigm's accelerator.

References

[1] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkuehler, and George Drettakis. 3D Gaussian Splatting for Real-Time Radiance Field Rendering. ACM Transactions on Graphics, 2023. https://repo-sam.inria.fr/fungraph/3d-gaussian-splatting/

[2] Samuli Laine, Janne Hellsten, Tero Karras, Yeongho Seol, Jaakko Lehtinen, and Timo Aila. Modular Primitives for High-Performance Differentiable Rendering. ACM Transactions on Graphics, 2020. https://nvlabs.github.io/nvdiffrast/

[3] Vickie Ye, Ruilong Li, Justin Kerr, Angjoo Kanazawa, and Matthew Tancik. gsplat: An Open-Source Library for Gaussian Splatting. arXiv:2409.06765, 2024. https://arxiv.org/abs/2409.06765

[4] Zehao Yu, Anpei Chen, Binbin Huang, Torsten Sattler, and Andreas Geiger. Mip-Splatting: Alias-Free 3D Gaussian Splatting. CVPR, 2024. https://openaccess.thecvf.com/content/CVPR2024/html/Yu_Mip-Splatting_Alias-free_3D_Gaussian_Splatting_CVPR_2024_paper.html

[5] Zhiwen Yan, Weng Fei Low, Yu Chen, and Gim Hee Lee. Multi-Scale 3D Gaussian Splatting for Anti-Aliased Rendering. CVPR, 2024. https://cvpr.thecvf.com/virtual/2024/poster/30347

[6] Zhihao Liang, Qi Zhang, Wenbo Hu, Ying Feng, Lei Zhu, and Kui Jia. Analytic-Splatting: Anti-Aliased 3D Gaussian Splatting via Analytic Integration. arXiv:2403.11056, 2024. https://arxiv.org/abs/2403.11056

[7] Michael Steiner, Thomas Koehler, Lukas Radl, Felix Windisch, Dieter Schmalstieg, and Markus Steinberger. AAA-Gaussians: Anti-Aliased and Artifact-Free 3D Gaussian Rendering. arXiv preprint, 2025. https://tugraz.elsevierpure.com/en/publications/aaa-gaussians-anti-aliased-and-artifact-free-3d-gaussian-renderin/

[8] Zhenya Yang, Bingchen Gong, and Kai Chen. LOD-GS: Level-of-Detail-Sensitive 3D Gaussian Splatting for Detail Conserved Anti-Aliasing. arXiv:2507.00554, 2025. https://papers.cool/arxiv/2507.00554

[9] Taewoo Suh, Sungpyo Kim, Jongmin Park, and Munchurl Kim. AA-Splat: Anti-Aliased Feed-forward Gaussian Splatting. arXiv:2603.29394, 2026. https://papers.cool/arxiv/2603.29394

[10] David Bauer, Qi Wu, Hamid Gadirov, and Kwan Liu Ma. GSCache: Real-Time Radiance Caching for Volume Path Tracing Using 3D Gaussian Splatting. IEEE Transactions on Visualization and Computer Graphics, 2026. https://research.rug.nl/en/publications/gscache-real-time-radiance-caching-for-volume-path-tracing-using-/

[11] Thomas Mueller, Fabrice Rousselle, Jan Novak, and Alexander Keller. Real-Time Neural Radiance Caching for Path Tracing. ACM Transactions on Graphics, 2021. https://research.nvidia.com/labs/rtr/publication/muller2021nrc/

[12] Thomas Mueller, Fabrice Rousselle, Alexander Keller, and Jan Novak. Neural Control Variates. ACM Transactions on Graphics, 2020. https://research.nvidia.com/labs/rtr/publication/muller2020ncv/

[13] Christoph Schied, Anton Kaplanyan, Chris Wyman, Anjul Patney, Chakravarty R. Alla Chaitanya, John Burgess, Shiqiu Liu, Carsten Dachsbacher, Aaron Lefohn, and Marco Salvi. Spatiotemporal Variance-Guided Filtering: Real-Time Reconstruction for Path-Traced Global Illumination. High Performance Graphics, 2017. https://research.nvidia.com/labs/rtr/publication/schied2017spatiotemporal/

[14] Steve Bako, Thijs Vogels, Brian McWilliams, Mark Meyer, Jan Novak, Alex Harvill, Pradeep Sen, Tony DeRose, and Fabrice Rousselle. Kernel-Predicting Convolutional Networks for Denoising Monte Carlo Renderings. ACM Transactions on Graphics, 2017. https://studios.disneyresearch.com/2017/07/20/kernel-predicting-convolutional-networks-for-denoising-monte-carlo-renderings/

[15] Benedikt Bitterli, Chris Wyman, Matt Pharr, Peter Shirley, Aaron Lefohn, and Wojciech Jarosz. Spatiotemporal Reservoir Resampling for Real-Time Ray Tracing with Dynamic Direct Lighting. ACM Transactions on Graphics, 2020. https://research.nvidia.com/labs/rtr/publication/bitterli2020spatiotemporal/

[16] Shakiba Kheradmand, Delio Vicini, George Kopanas, Dmitry Lagun, Kwang Moo Yi, Mark Matthews, and Andrea Tagliasacchi. StochasticSplats: Stochastic Rasterization for Sorting-Free 3D Gaussian Splatting. arXiv:2503.24366, 2025. https://arxiv.org/abs/2503.24366

[17] Yang Zhou, Songyin Wu, and Ling-Qi Yan. Unified Gaussian Primitives for Scene Representation and Rendering. arXiv:2406.09733, 2024. https://mangosister.github.io/gsr_site/

[18] Guanjun Wu, Taoran Yi, Jiemin Fang, Lingxi Xie, Xiaopeng Zhang, Wei Wei, Wenyu Liu, Qi Tian, and Xinggang Wang. 4D Gaussian Splatting for Real-Time Dynamic Scene Rendering. CVPR, 2024. https://guanjunwu.github.io/4dgs/

[19] Zhan Li, Zhang Chen, Zhong Li, and Yi Xu. Spacetime Gaussian Feature Splatting for Real-Time Dynamic View Synthesis. arXiv:2312.16812, 2023. https://arxiv.org/abs/2312.16812

[20] Yuanxing Duan, Fangyin Wei, Qiyu Dai, Yuhang He, Wenzheng Chen, and Baoquan Chen. 4D-Rotor Gaussian Splatting: Towards Efficient Novel View Synthesis for Dynamic Scenes. 2024. https://research.nvidia.com/publication/2024-02_4d-rotor-gaussian-splatting-towards-efficient-novel-view-synthesis-dynamic

[21] Youtian Lin, Zuozhuo Dai, Siyu Zhu, and Yao Yao. Gaussian-Flow: 4D Reconstruction with Dynamic 3D Gaussian Particle. arXiv:2312.03431, 2023. https://huggingface.co/papers/2312.03431

[22] Quankai Gao, Qiangeng Xu, Zhe Cao, Ben Mildenhall, Wenchao Ma, Le Chen, Dahang Tang, and Ulrich Neumann. GaussianFlow: Splatting Gaussian Dynamics for 4D Content Creation. arXiv:2403.12365, 2024. https://huggingface.co/papers/2403.12365

[23] Ruijie Zhu, Yanzhe Liang, Hanzhi Chang, Jiacheng Deng, Jiahao Lu, Wenfei Yang, Tianzhu Zhang, and Yongdong Zhang. MotionGS: Exploring Explicit Motion Guidance for Deformable 3D Gaussian Splatting. arXiv:2410.07707, 2024. https://huggingface.co/papers/2410.07707

[24] Qingyuan Tang, Yufei Yin, Yanming Zhu, Zhou Yu, Zhenzhong Kuang, Jiajun Ding, and Jifa He. KF-GS: Kalman Filter-Guided Gaussian Splatting for Real-Time High-Quality Dynamic Scene Reconstruction. Journal of Visual Communication and Image Representation, in press, 2026. https://www.sciencedirect.com/science/article/pii/S1047320326001100

[25] Xuezhen Wang, Li Ma, Yulin Shen, Zeyu Wang, and Pedro V. Sander. RetimeGS: Continuous-Time Reconstruction of 4D Gaussian Splatting. CVPR, 2026. https://william-wang2.github.io/RetimeGS/

[26] NVIDIA. NVIDIA DLSS 5 Delivers AI-Powered Breakthrough in Visual Fidelity for Games. March 16, 2026. https://nvidianews.nvidia.com/news/nvidia-dlss-5-delivers-ai-powered-breakthrough-in-visual-fidelity-for-games

[27] NVIDIA. DLSS 4.5 Dynamic Multi Frame Generation and Multi Frame Generation 6X Available Now. March 31, 2026. https://www.nvidia.com/en-us/geforce/news/dlss-4-5-dynamic-multi-frame-generation-6x-mode-released/

[28] NVIDIA Developer. NVIDIA DLSS. Accessed May 5, 2026. https://developer.nvidia.com/rtx/dlss

[29] NVIDIA. NVIDIA DLSS 4 Introduces Multi Frame Generation and Enhancements for All DLSS Technologies. January 6, 2025. https://www.nvidia.com/en-sg/geforce/news/dlss4-multi-frame-generation-ai-innovations/

[30] NVIDIA Research. DLSS 4: Transforming Real-Time Graphics with AI. Technical report, 2025. https://research.nvidia.com/labs/adlr/DLSS4/

[31] NVIDIA. NVIDIA Hopper Architecture In-Depth. Technical blog, 2022. https://developer.nvidia.com/blog/nvidia-hopper-architecture-in-depth/

[32] NVIDIA. FP8 Current Scaling. Transformer Engine documentation, accessed May 5, 2026. https://docs.nvidia.com/deeplearning/transformer-engine/user-guide/features/low_precision_training/fp8_current_scaling/fp8_current_scaling.html

[33] NVIDIA. CUDA Math API Reference Manual. CUDA Toolkit documentation, accessed May 5, 2026. https://docs.nvidia.com/cuda/archive/13.0.1/cuda-math-api/index.html

[34] NVIDIA. NVFP4. Transformer Engine documentation, accessed May 5, 2026. https://nvidia.github.io/TransformerEngine/features/low_precision_training/nvfp4/nvfp4.html

[35] Open Compute Project. OCP Microscaling Formats (MX) Specification, Version 1.0. 2026. https://www.opencompute.org/documents/ocp-microscaling-formats-mx-v1-0-spec-final-pdf

[36] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient Memory Management for Large Language Model Serving with PagedAttention. SOSP, 2023. https://huggingface.co/papers/2309.06180

[37] Ramya Prabhu, Ajay Nayak, Jayashree Mohan, Ramachandran Ramjee, and Ashish Panwar. vAttention: Dynamic Memory Management for Serving LLMs without PagedAttention. ASPLOS, 2025. https://www.microsoft.com/en-us/research/publication/vattention-dynamic-memory-management-for-serving-llms-without-pagedattention/

[38] Intel. Intel XeSS Super Resolution Developer Guide 2.0. Accessed May 5, 2026. https://www.intel.com/content/www/us/en/developer/articles/technical/xess-sr-developer-guide.html

[39] AMD. AMD FSR Technologies. Accessed May 5, 2026. https://www.amd.com/en/products/graphics/technologies/fidelityfx/super-resolution.html